CYBERSECURITY INTERNSHIP

Hands-On Lab Report

Penetration testing, web application security, vulnerability assessment and incident reporting

Agamya Cyber Tech | July - October 2026

Environment: Kali Linux (VirtualBox), Metasploitable, DVWA, OWASP Juice Shop, OpenVAS / GVM
Tools: Nmap, Metasploit, Burp Suite, Wireshark, macchanger, arpspoof, dsniff

All exercises were performed on personal virtual machines and intentionally vulnerable training targets. No client or real-world systems were tested. For educational purposes only.

Contents

1. Linux fundamentals 3
2. Network scanning with Nmap 8
3. OWASP Top 10 study notes 11
4. FTP exploitation lab: vsftpd 2.3.4 with Metasploit 12
5. Core security concepts 15
6. DVWA setup and brute force (Burp Suite Intruder) 16
7. DVWA: command injection 24
8. DVWA: CSRF 26
9. DVWA: file inclusion (LFI / RFI) 30
10. DVWA: unrestricted file upload 32
11. DVWA: reflected XSS 34
12. DVWA: stored XSS 35
13. DVWA: information disclosure via phpinfo() 36
14. Spoofing: concepts and lab demonstrations 38
15. Vulnerability scanning with OpenVAS / GVM 43
16. OWASP Juice Shop challenges 46
17. Incident report: PHP2Shell 48

Scope and ethics. Every exercise below was run against my own virtual machines or intentionally vulnerable training targets (Metasploitable, DVWA, Juice Shop, `scanme.nmap.org`, `localhost`). No client systems or real-world targets were tested. The techniques are documented for learning and defensive understanding only.

1. Linux fundamentals

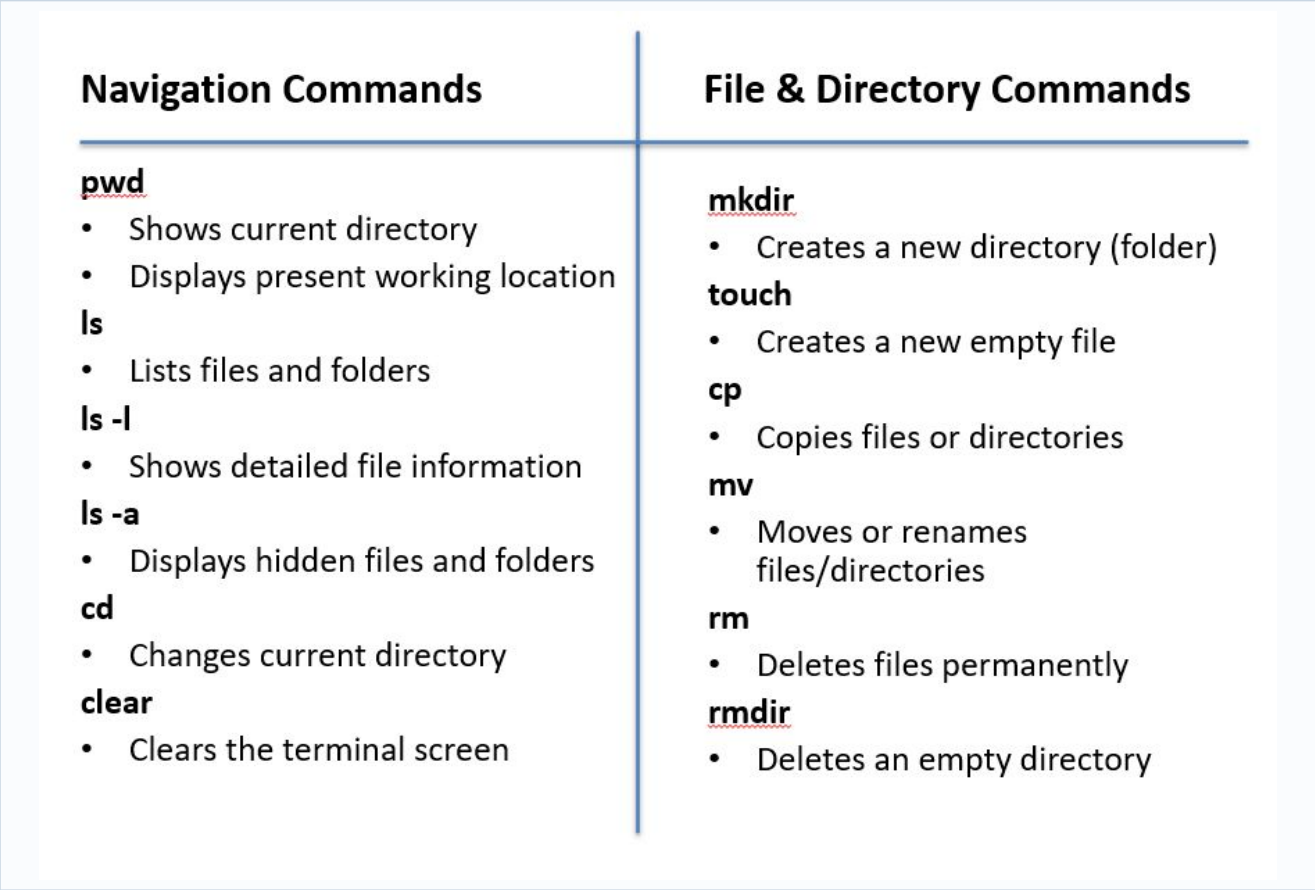
Date: 24 Jul 2026

Practiced the command-line skills needed for security work, grouped as quick-reference cheat sheets:

Category	Commands
Navigation	`pwd`, `ls`, `ls -l`, `ls -a`, `cd`, `clear`
Files and directories	`mkdir`, `touch`, `cp`, `mv`, `rm`, `rmdir`
Viewing	`cat`, `less`, `head`, `tail`
Searching	`find`, `grep`, `which`
Permissions	`chmod`, `chown`, `sudo`
System info	`whoami`, `id`, `uname -a`, `hostname`, `df -h`, `free -h`
Processes and packages	`ps`, `top`, `kill`, `apt update/install/upgrade`
Networking and security	`ip a`, `ping`, `curl`, `wget`, `ssh`, `netstat`/`ss`, `nmap`, `openssl`, `tar`, `zip`

Hands-on practice in Kali (creating, copying, renaming and searching files):

Evidence from the lab



Navigation Commands

pwd
- Shows current directory
- Displays present working location

ls
- Lists files and folders

ls -l
- Shows detailed file information

ls -a
- Displays hidden files and folders

cd
- Changes current directory

clear
- Clears the terminal screen

File & Directory Commands

mkdir
- Creates a new directory (folder)

touch
- Creates a new empty file

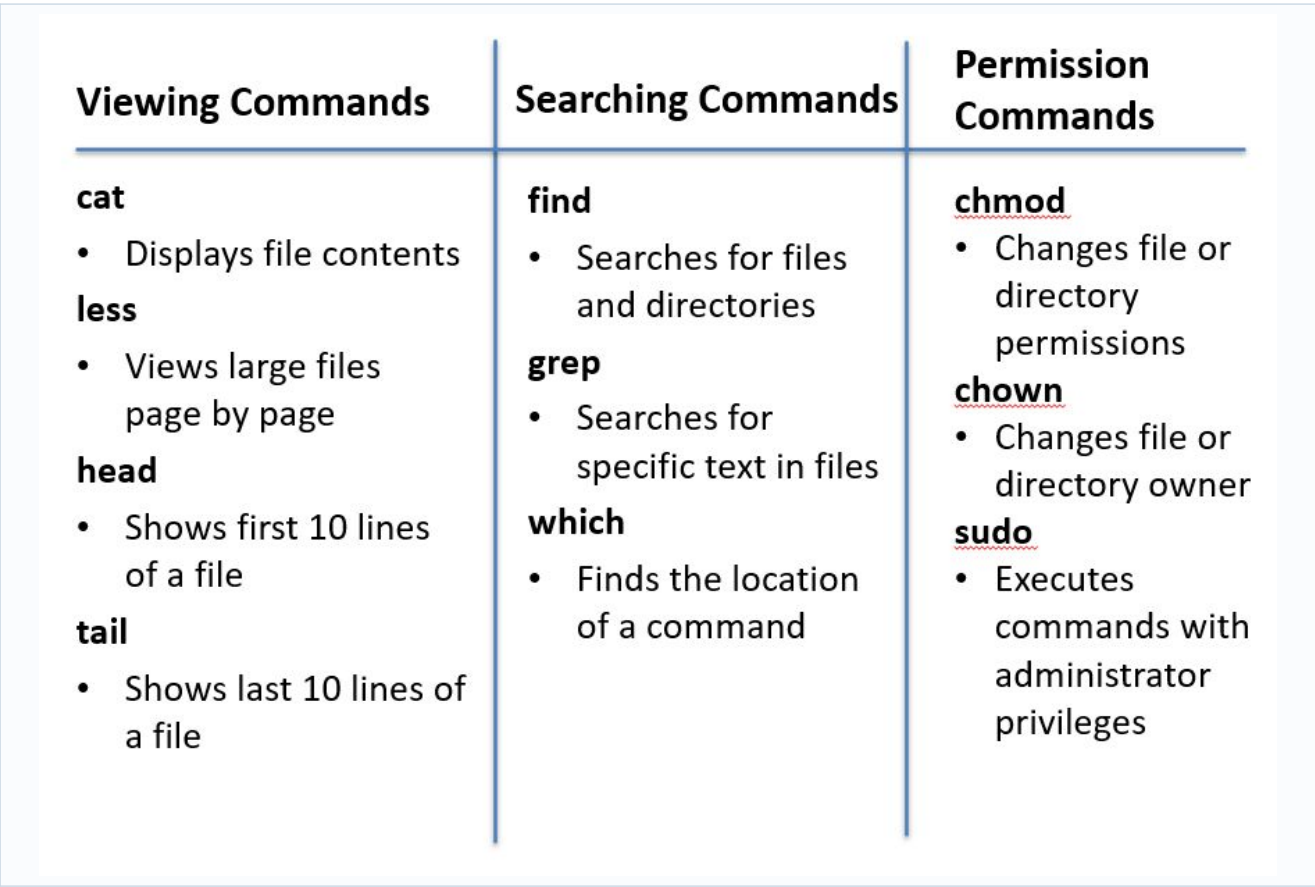
cp
- Copies files or directories

mv
- Moves or renames files/directories

rm
- Deletes files permanently

rmdir
- Deletes an empty directory

Figure 1. Cheat sheet: navigation and file/directory commands



Viewing Commands

cat
- Displays file contents

less
- Views large files page by page

head
- Shows first 10 lines of a file

tail
- Shows last 10 lines of a file

Searching Commands

find
- Searches for files and directories

grep
- Searches for specific text in files

which
- Finds the location of a command

Permission Commands

chmod
- Changes file or directory permissions

chown
- Changes file or directory owner

sudo
- Executes commands with administrator privileges

Figure 2. Cheat sheet: viewing, searching and permission commands

System Information Commands

whoami
- Displays current username

id
- Shows user ID and group IDs

uname -a
- Displays system and kernel information

hostname
- Shows the system's hostname

df -h
- Displays disk space usage in human-readable format

free -h
- Shows memory (RAM) usage in human-readable format

Process Management Commands

ps
- Lists running processes

top
- Displays real-time system and process information

kill
- Terminates a running process

Package Management

sudo apt update
- Updates package list

sudo apt install <package>
- Installs a package

sudo apt upgrade
- Upgrades installed packages

Figure 3. Cheat sheet: system information, process and package commands

Networking Commands

ip a
- Displays IP address and network interfaces

ping
- Tests network connectivity

curl
- Transfers data from or to a server

wget
- Downloads files from the internet

ssh
- Connects to a remote system securely

netstat / ss
- Shows active network connections and ports

Cybersecurity Commands

nmap
- Scans networks and discovers devices

openssl
- Performs encryption and SSL/TLS operations

tar
- Archives and extracts files

zip
- Compresses files and folders

Figure 4. Cheat sheet: networking and security commands

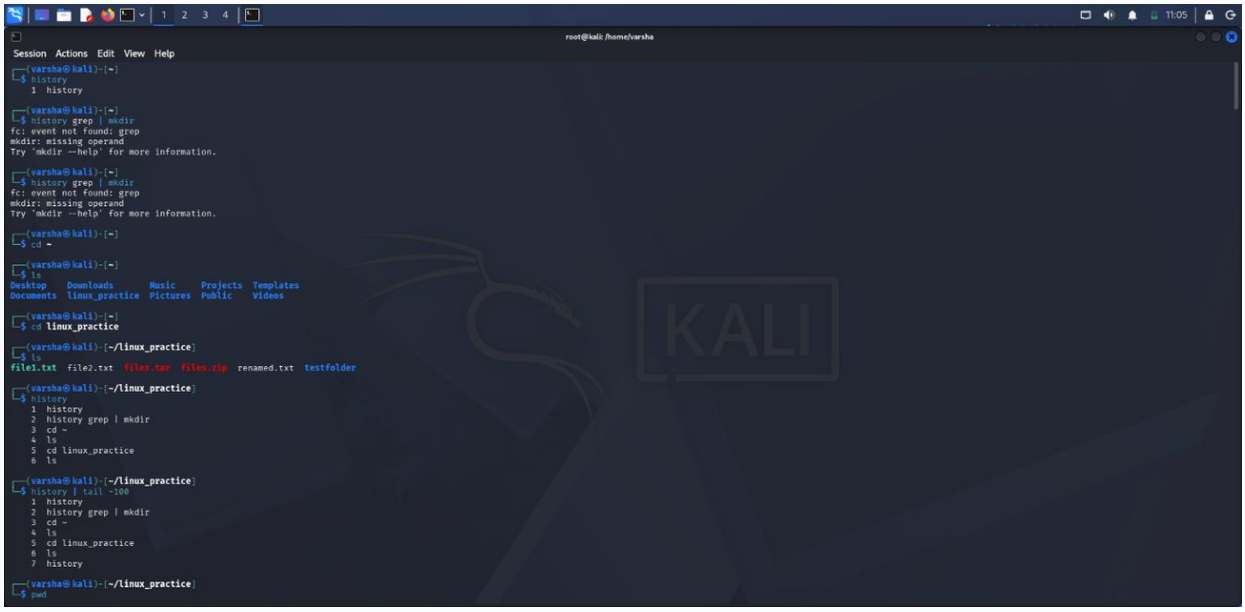


Figure 5. Terminal practice in Kali: history and command basics

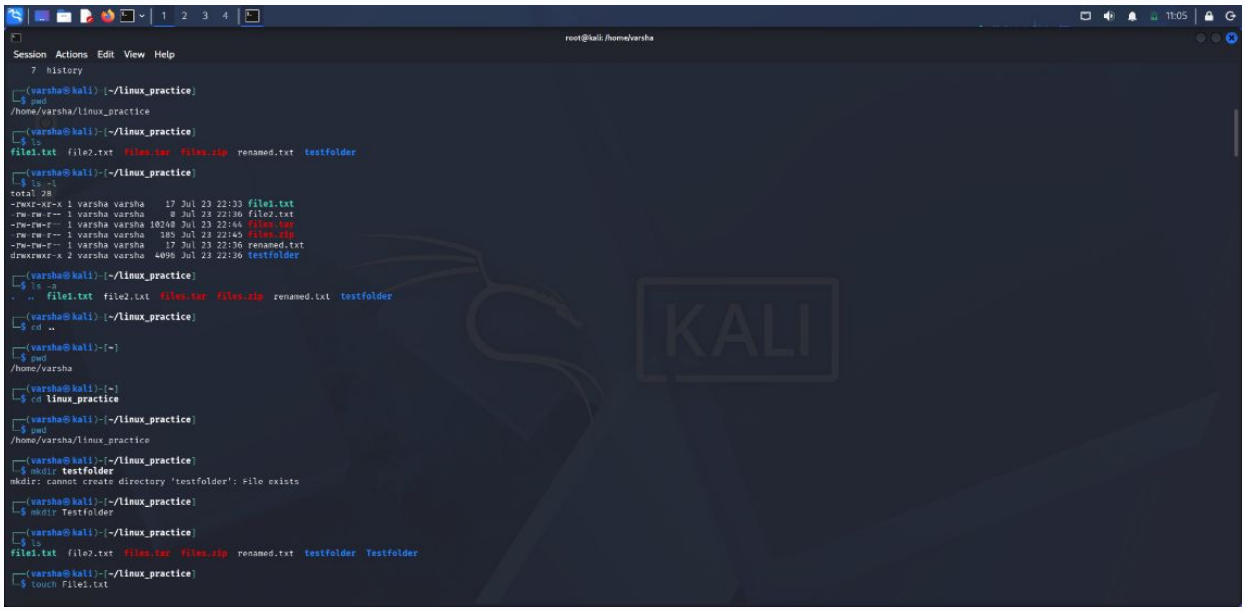


Figure 6. Terminal practice: pwd, ls and ls -l in the practice folder

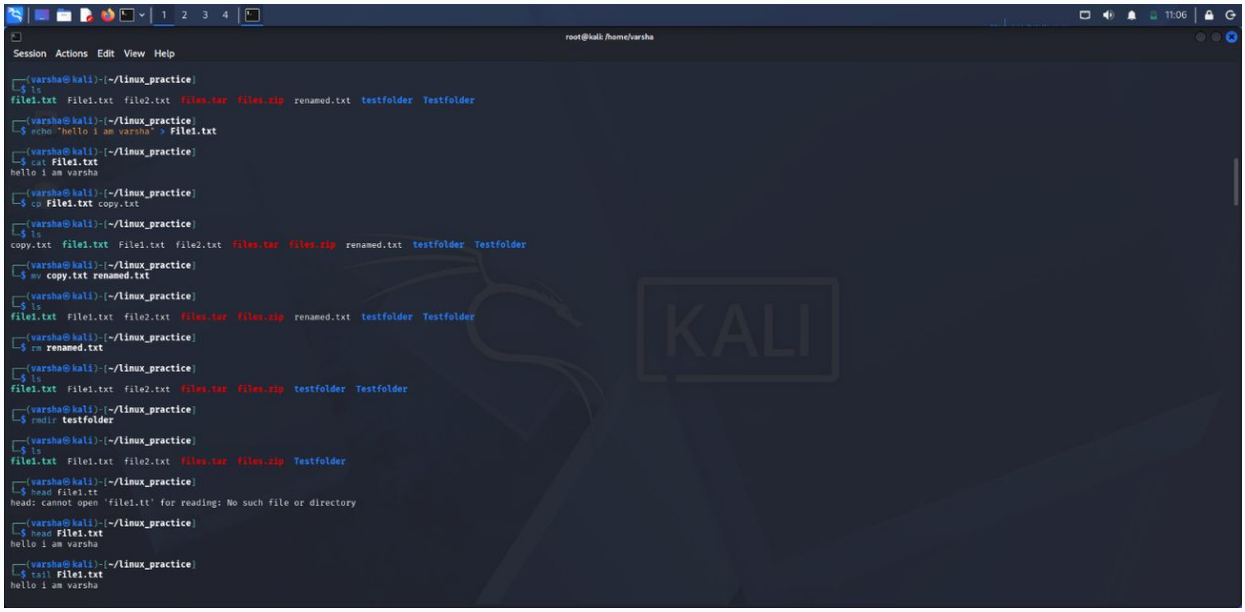


Figure 7. Terminal practice: echo and cat on a practice file

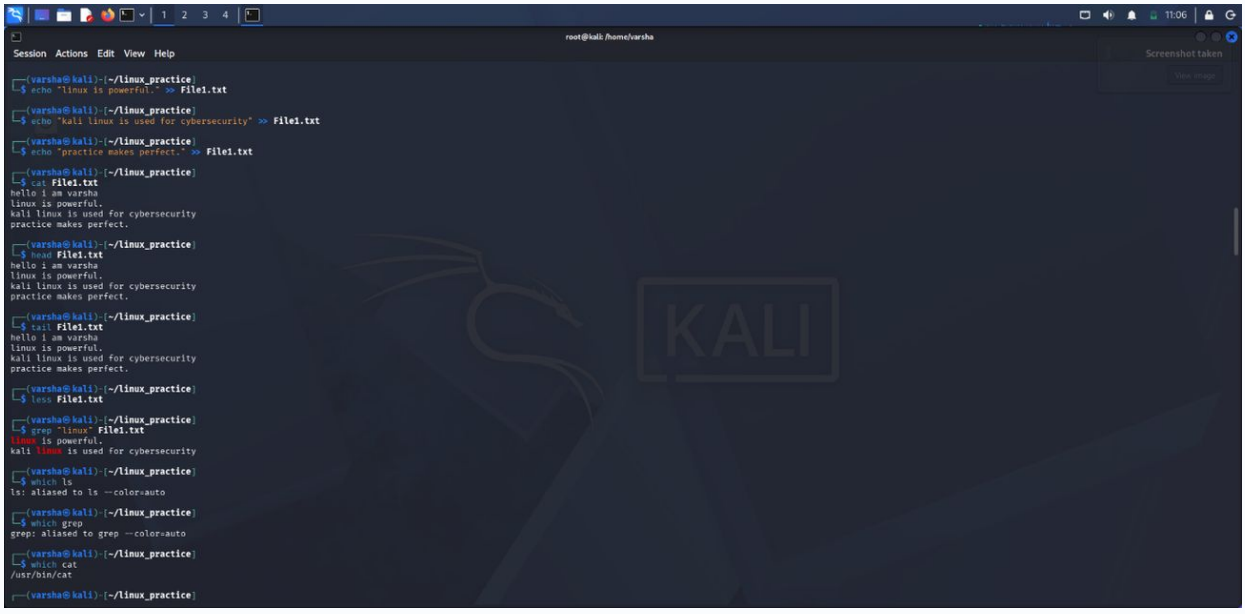


Figure 8. Terminal practice: appending lines to a file with echo >>

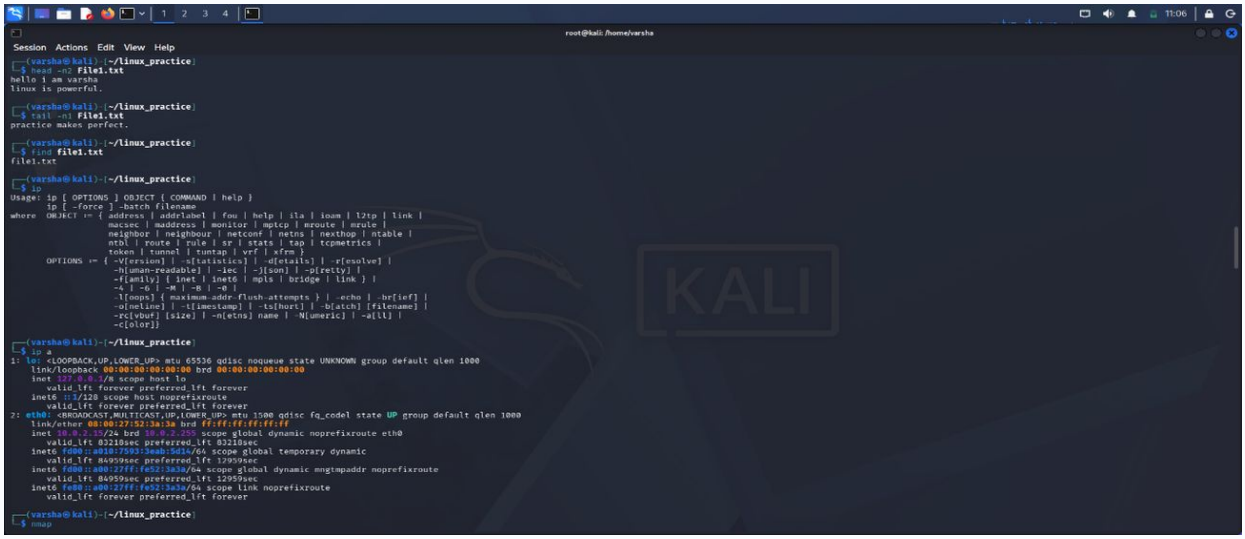


Figure 9. Terminal practice: head -n 2 and tail -n 1 on a practice file

2. Network scanning with Nmap

Date: 25 Jul 2026 · **Tools:** Nmap 7.99 on Kali Linux (VirtualBox)

Aim: Learn Nmap scan types and use them for host discovery, port enumeration, and service detection.

Technique	Command	What it shows
Host discovery (ping scan)	`nmap -sn 10.0.2.0-255`	3 live hosts found on the lab network
Default scan of a range	`nmap 10.0.2.0-255`	One host with ports 80 (HTTP), 135 (MSRPC), 445 (SMB) open; 992 ports filtered
Scan a single host	`nmap 192.168.70.129`	Host up, all scanned ports filtered (firewall)
Selected ports	`nmap -p 80,443,25,53 192.168.70.129`	Only the listed ports scanned
All 65,535 TCP ports	`nmap -p- 192.168.70.129`	Full TCP scan, much longer runtime
Localhost / loopback	`nmap localhost`, `nmap 127.0.0.1`	1000 closed ports, nothing exposed locally
TCP SYN (stealth) scan	`sudo nmap -sS localhost`	Detects open ports without completing the handshake
UDP scan	`sudo nmap -sU localhost`	Closed/filtered UDP ports
Subnet sweep	`nmap 192.168.1.0/24`	256 addresses probed
Public test target	`nmap scanme.nmap.org`	Scan against the host the Nmap project provides for practice

Takeaway: "Filtered" results usually mean a firewall is dropping probes, while "closed" means the host answered but nothing is listening. Reading that difference correctly matters when judging exposure.

Evidence from the lab

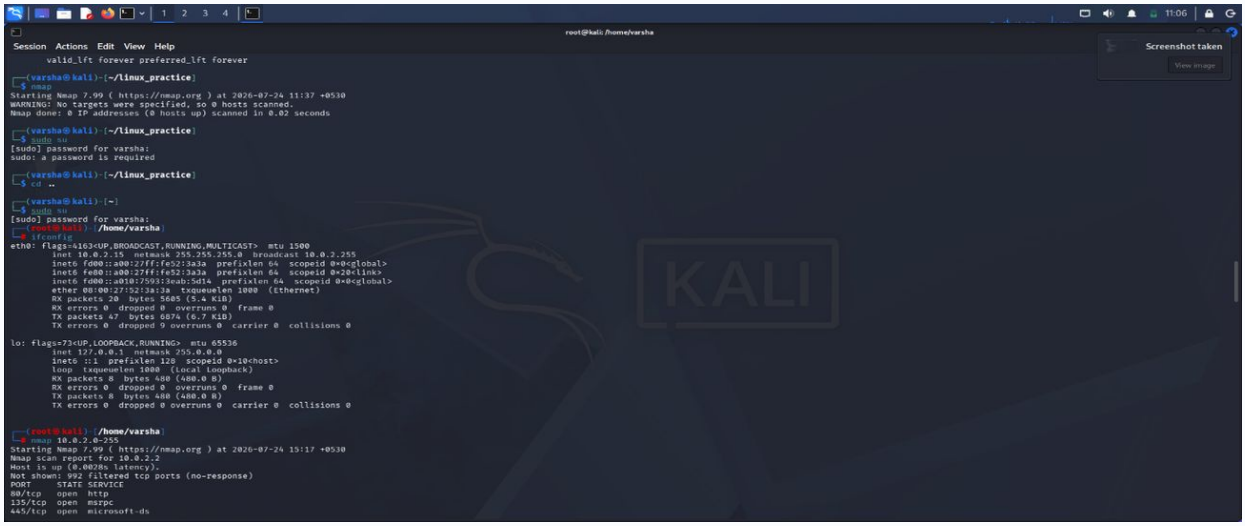


Figure 10. nmap with no target: "No targets were specified, so 0 hosts scanned"

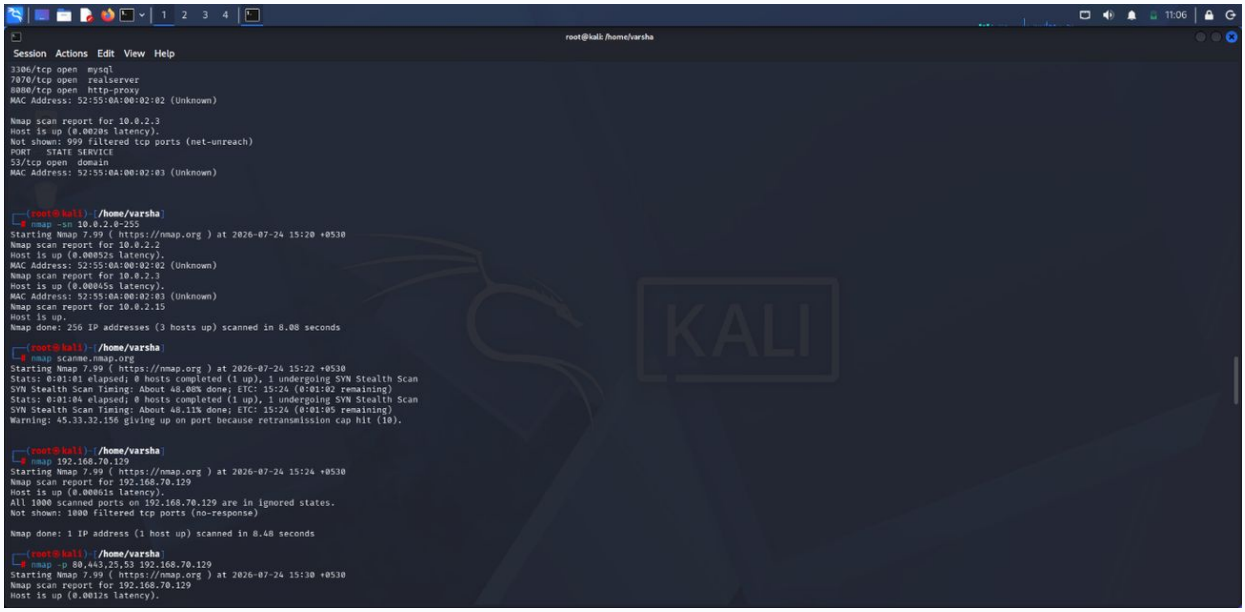


Figure 11. Scan results listing open ports and services on the lab network

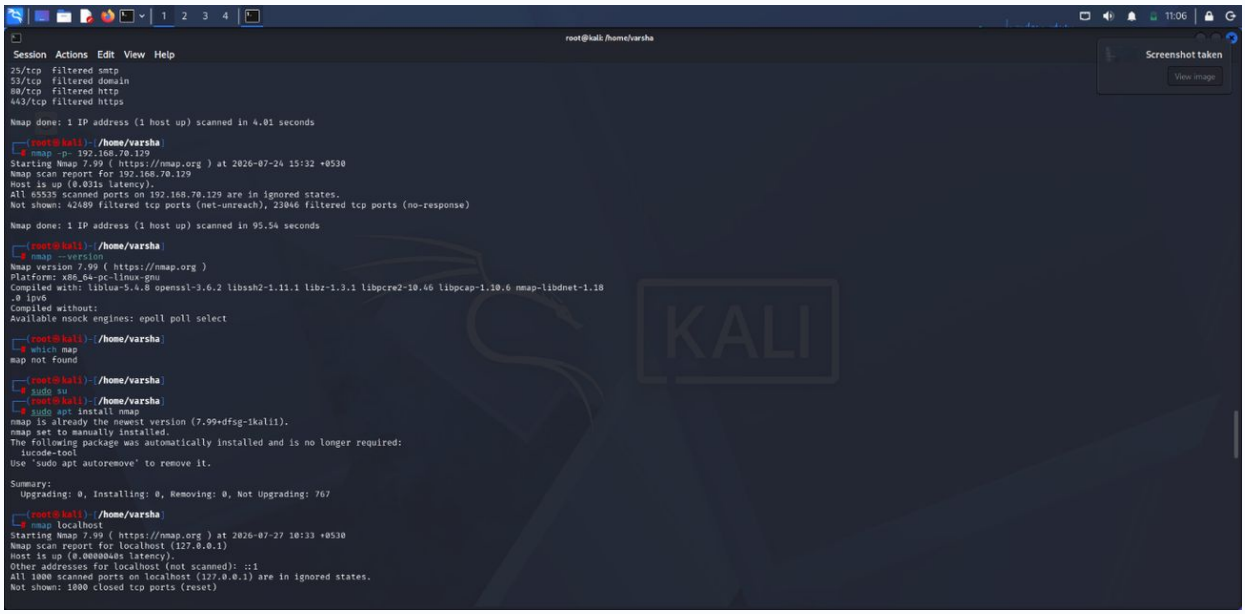


Figure 12. Selected-port scan: ports 25, 53, 80 and 443 reported as filtered

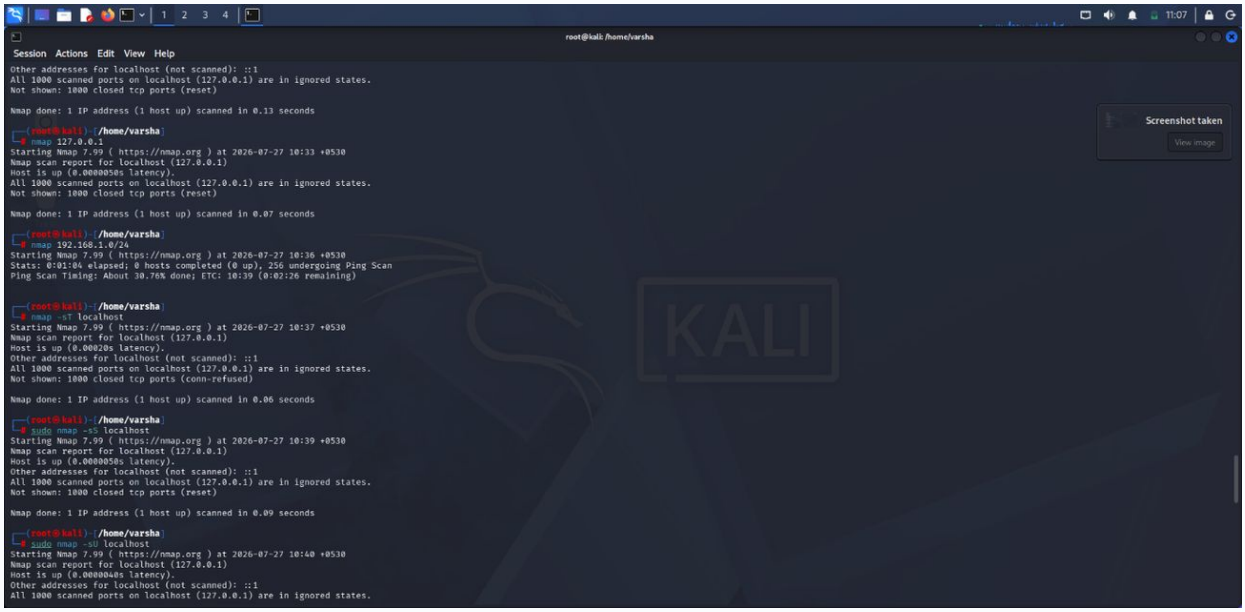


Figure 13. Localhost scan: 1000 closed TCP ports

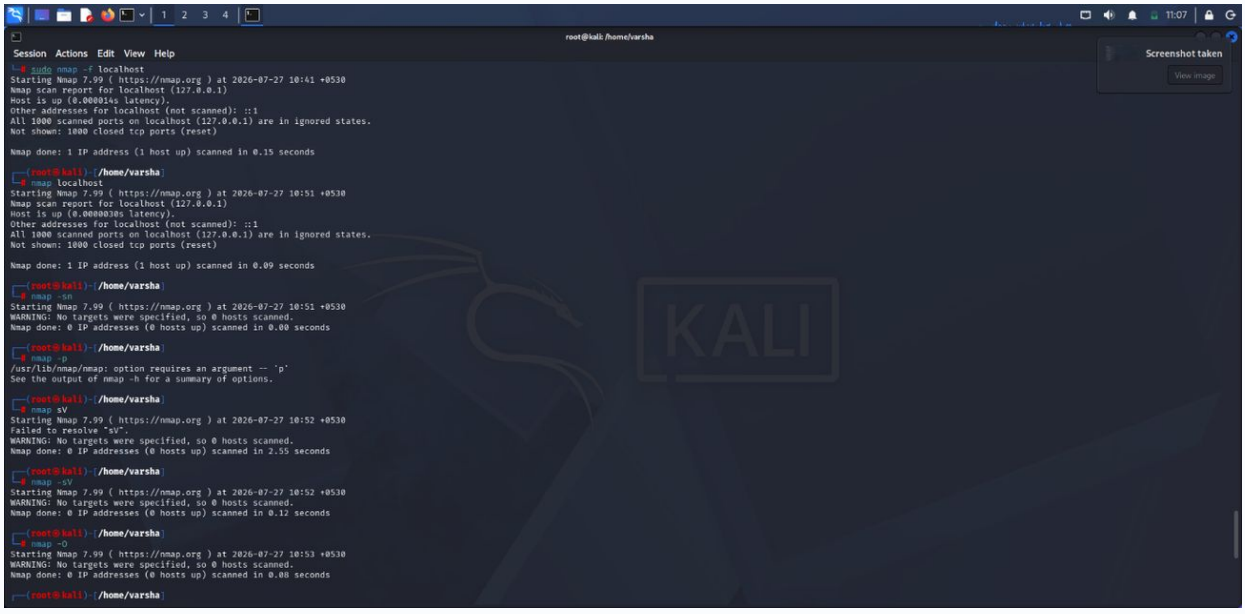


Figure 14. Localhost scan with additional scan options

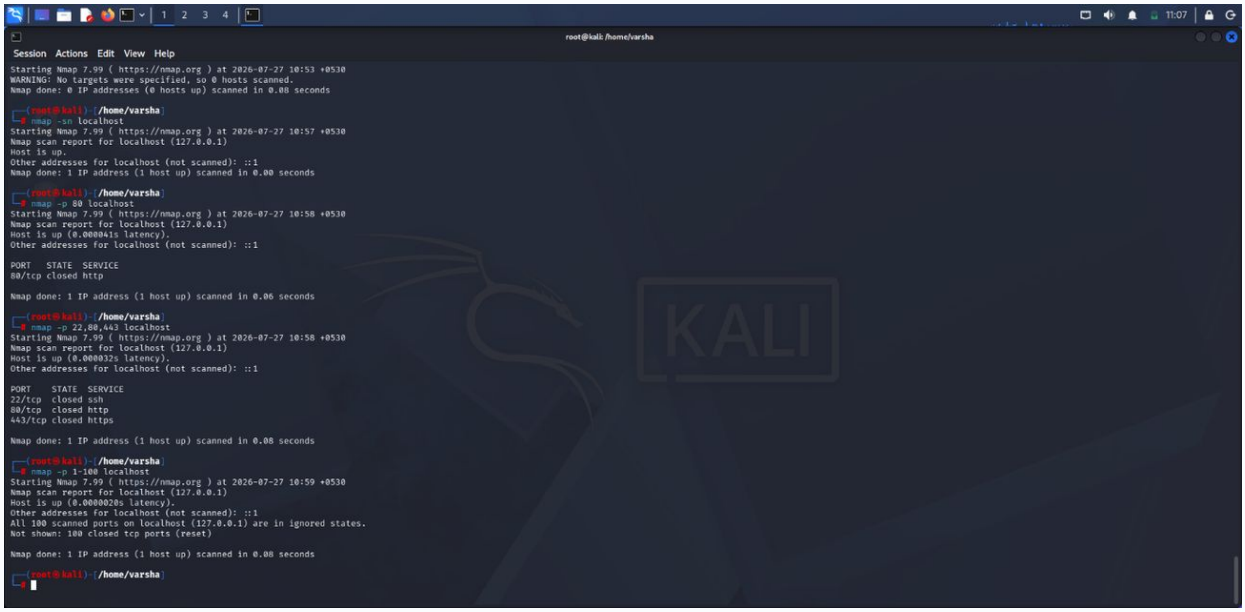


Figure 15. Host-discovery (-sn) scan output

3. OWASP Top 10 study notes

Date: 27 Jul 2026

Wrote structured notes for each category: definition, causes, real-world example, impact, prevention, and testing tools.

ID	Category	Core prevention	Example tools
A01	Broken Access Control	Server-side authorization, least privilege, RBAC	Burp Suite, ZAP, Postman
A02	Cryptographic Failures	HTTPS, strong password hashing, modern crypto	SSL Labs, Burp Suite
A03	Injection	Parameterized queries, input validation	Burp Suite, ZAP, SQLMap (authorized only)
A04	Insecure Design	Threat modeling, secure SDLC, design reviews	Threat Modeling Tool, OWASP ASVS
A05	Security Misconfiguration	Remove defaults and unused services, audits	Nikto, Nmap, ZAP
A06	Vulnerable and Outdated Components	Update dependencies, monitor advisories	Dependency-Check, Snyk, Trivy
A07	Identification and Authentication Failures	Strong passwords, MFA, session management	Burp Suite, ZAP
A08	Software and Data Integrity Failures	Code signing, integrity checks, secure pipelines	Dependabot, Snyk
A09	Logging and Monitoring Failures	Enable logging, alerting, review	Splunk, ELK, Wazuh
A10	Server-Side Request Forgery (SSRF)	Validate and allowlist outbound URLs	Burp Suite

4. FTP exploitation lab: vsftpd 2.3.4 with Metasploit

Date: 28 Jul 2026 · **Attacker:** Kali Linux VM · **Target:** Metasploitable VM (lab network) · **Port:** 21/tcp

Objective: Identify the FTP service on an intentionally vulnerable machine, determine its version, and validate the known backdoor using Metasploit.

Method

1. **Reachability:** confirmed the target was alive with `fping`.
2. **Enumeration:** `nmap <target> -sV` identified `21/tcp open ftp vsftpd 2.3.4`, plus SSH, Telnet, HTTP, SMB, MySQL and PostgreSQL.
3. **Module search:** in `msfconsole`, `search vsftpd` returned `exploit/unix/ftp/vsftpd_234_backdoor`; selected it with `use`.
4. **Configuration:** reviewed `show options`, then set `RHOSTS` (target) and `LHOST` (Kali).
5. **Execution:** `run`. Metasploit reported the target as vulnerable and opened a command shell.
6. **Verification:** confirmed the user identity and browsed the target filesystem (`ls`) to prove access.

Finding: vsftpd 2.3.4 contains a well-known backdoor that gives remote command execution.

Remediation (what a real fix looks like): upgrade to a patched vsftpd release, remove or restrict FTP in favor of SFTP/FTPS, firewall port 21, and keep an inventory so outdated services get flagged.

Evidence from the lab

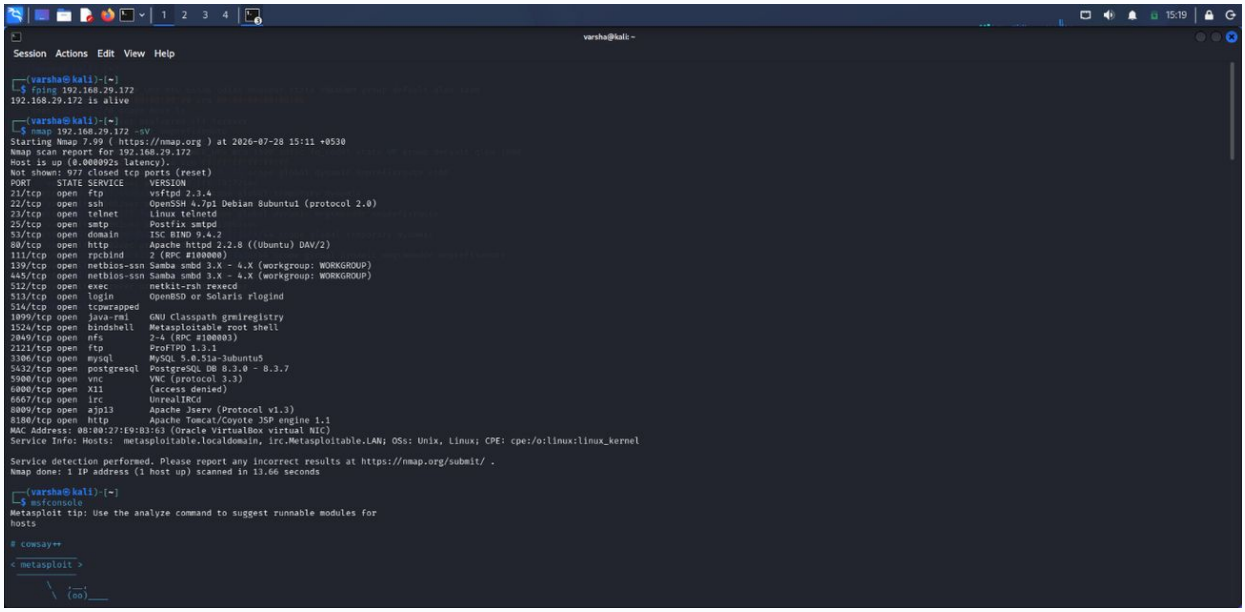


Figure 16. Nmap -sV identifies vsftpd 2.3.4 on port 21

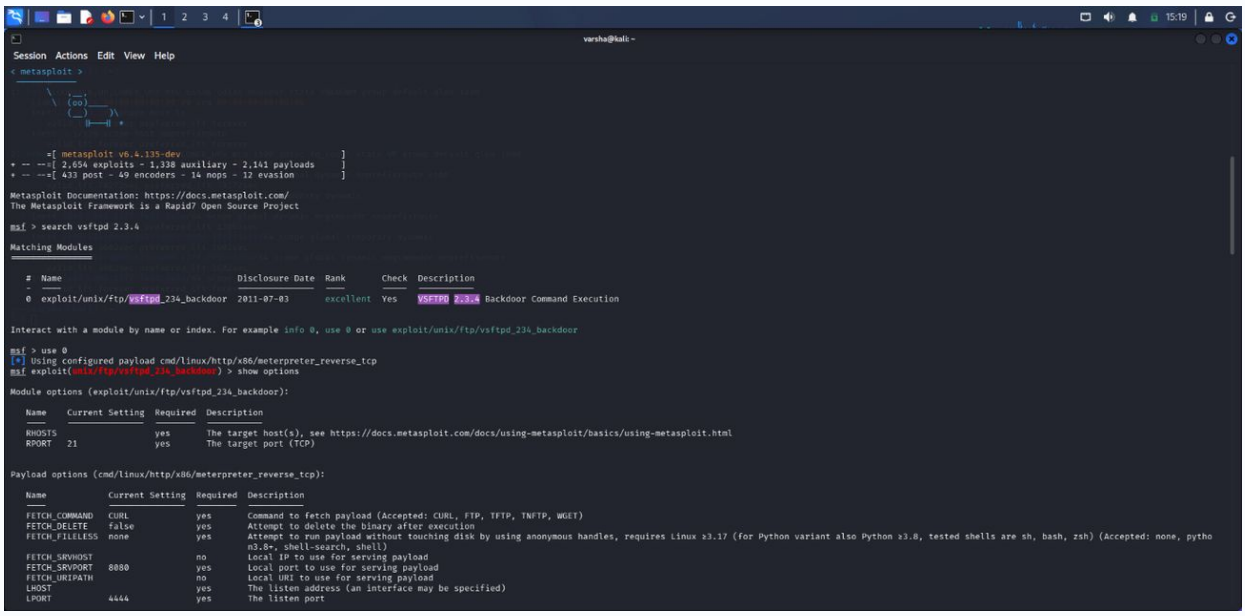


Figure 17. Searching and selecting the vsftpd_234_backdoor module

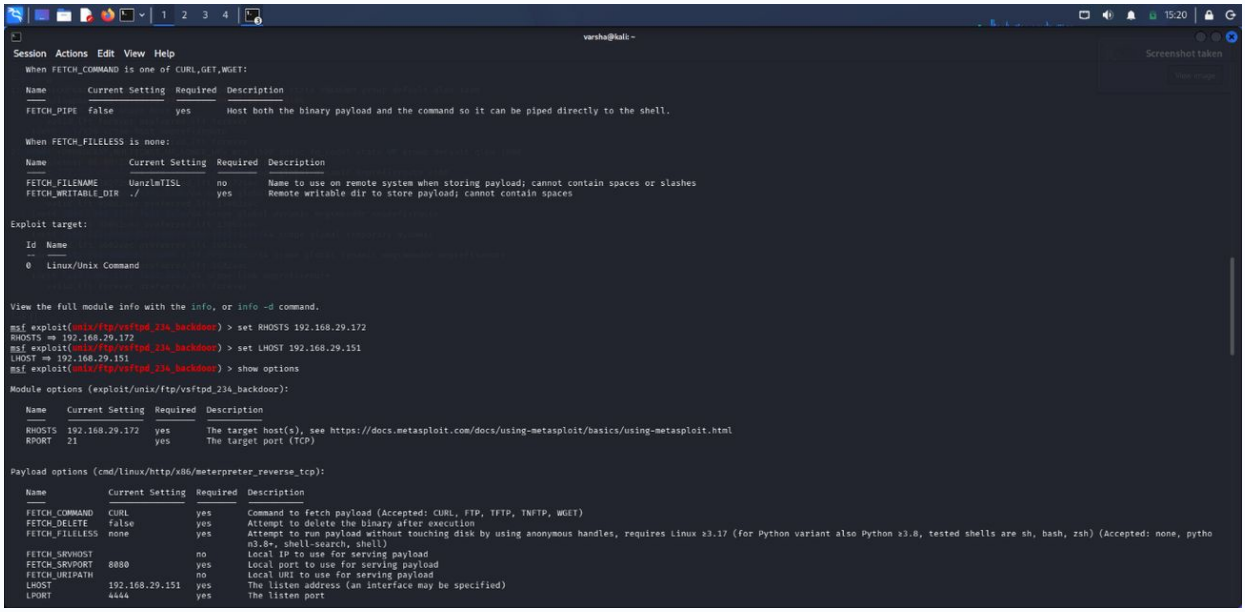


Figure 18. Configuring RHOSTS and LHOST

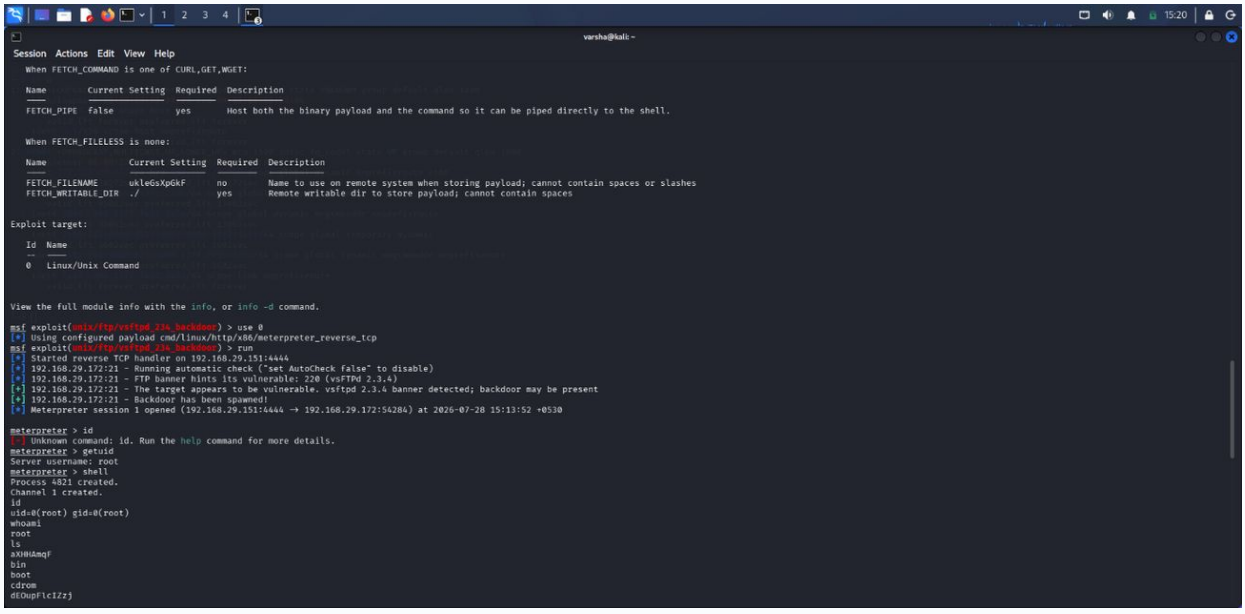


Figure 19. Running the module: target reported as vulnerable, shell opened

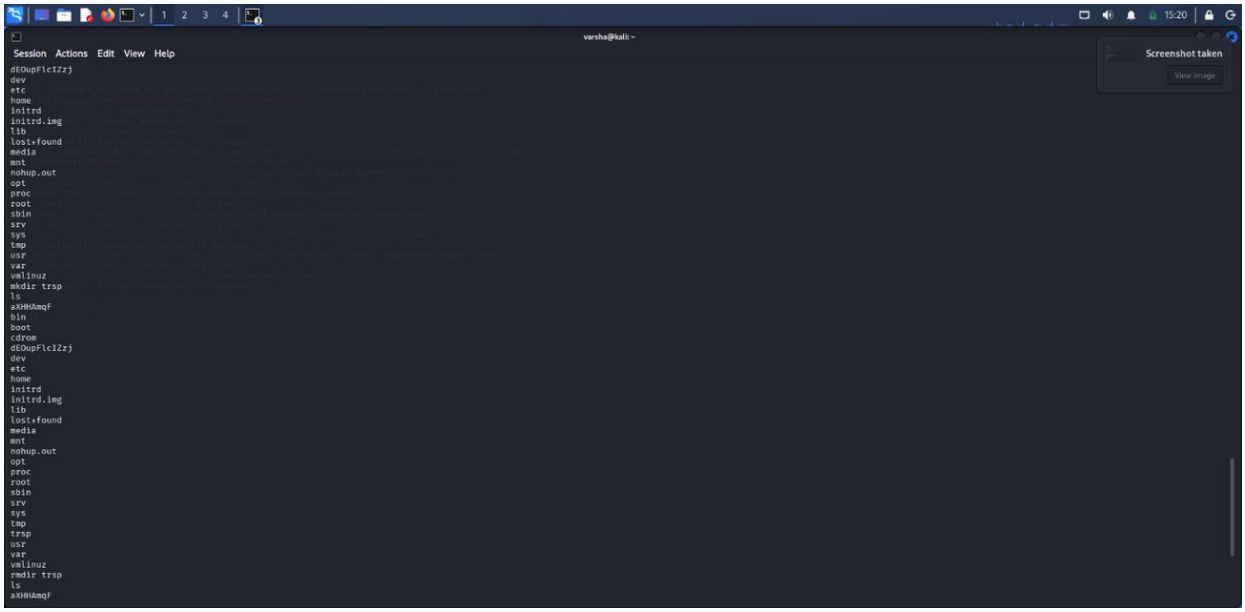


Figure 20. Checking identity and listing files on the target shell

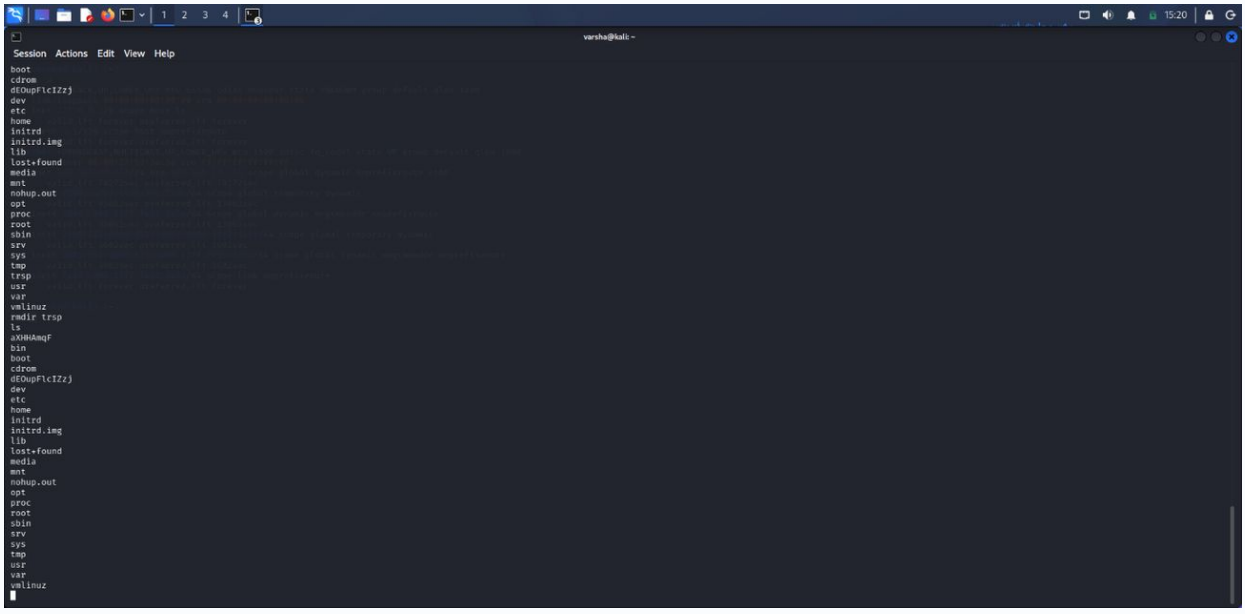


Figure 21. Verifying access to the target filesystem

5. Core security concepts

Date: 29 Jul 2026

Short notes written in my own words on 25 topics:

- **Network and protocols:** SMB, SMTP, ARP, RDP
- **Attacks:** ARP poisoning, session hijacking, phishing, SQL injection, XSS, CSRF, brute force, MITM, spoofing, DoS, DDoS
- **Access control (AAA):** authentication, authorization, accounting
- **Cryptography:** encryption, hashing, digital signatures
- **Defenses:** VPN, firewall, IDS, IPS

6. DVWA setup and brute force (Burp Suite Intruder)

Dates: 30 Jul 2026 · **Tools:** DVWA, Apache, MariaDB, PHP, Burp Suite, FoxyProxy

Installing DVWA on Kali

1. Cloned DVWA into `/var/www/html` (needed `sudo` due to permissions).
2. Copied `config.inc.php.dist` to `config.inc.php` and set database credentials.
3. Started and verified MariaDB; created the `dvwa` database and a dedicated user with privileges on that database only.
4. Started Apache, edited `php.ini` for DVWA's requirements, restarted Apache.
5. Opened the setup page, ran **Create / Reset Database**, and confirmed all modules loaded.

Lab credentials only (default DVWA values). Never reuse these anywhere real.

Brute force, three difficulty levels

Low: Intercepted the login request in Burp, sent it to **Intruder** with attack type **Cluster Bomb**, and used `http-default-users.txt` and `http-default-pass.txt` as payload sets. Sorting by response length exposed one outlier (~4740 vs ~4702 for failures), which was the valid login.

Medium: Same method, but the server now sleeps ~2 seconds after each failure, so the attack is slower. The successful attempt was still the response-length outlier (~4750).

High: Each login form carries an **anti-CSRF token** that changes on every page load, so a plain Intruder run fails. I built a Burp **session handling rule with a macro** that fetches a fresh token before each attempt, scoped it to Intruder only, and added a **Grep Match** for the string `incorrect` to flag failures. The one response without that string (length ~4829 vs ~4719) was the success.

Takeaway: Rate-limiting and tokens slow brute force but do not stop it. Account lockout, MFA, and monitoring for repeated failures are the real controls.

Evidence from the lab

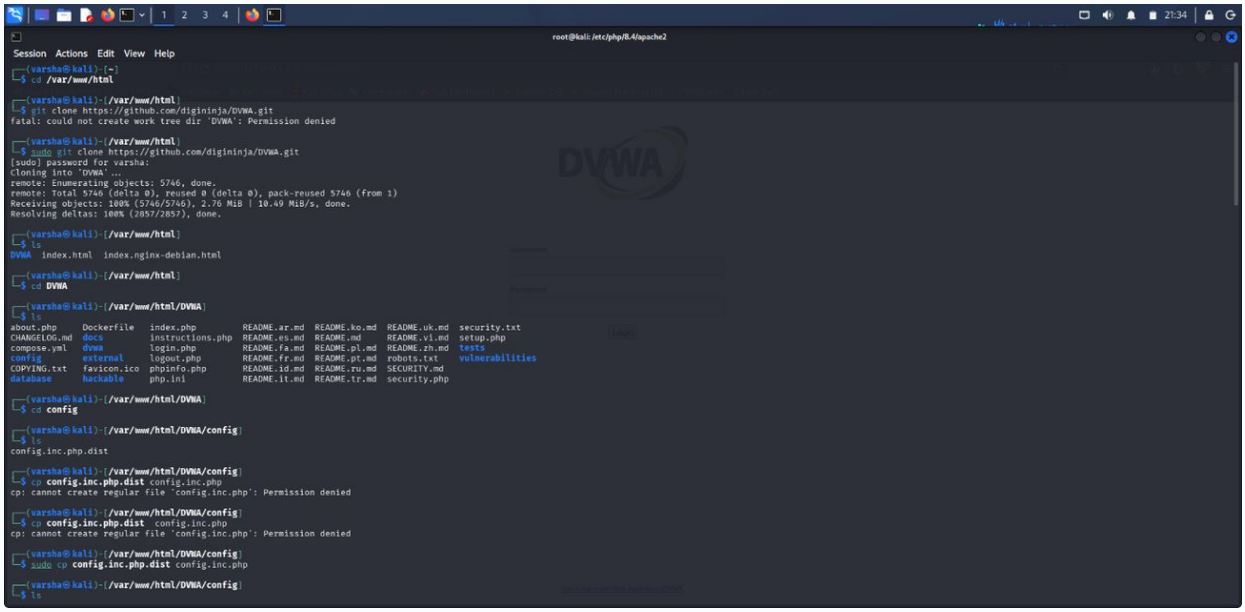


Figure 22. Cloning DVWA into /var/www/html (sudo required)

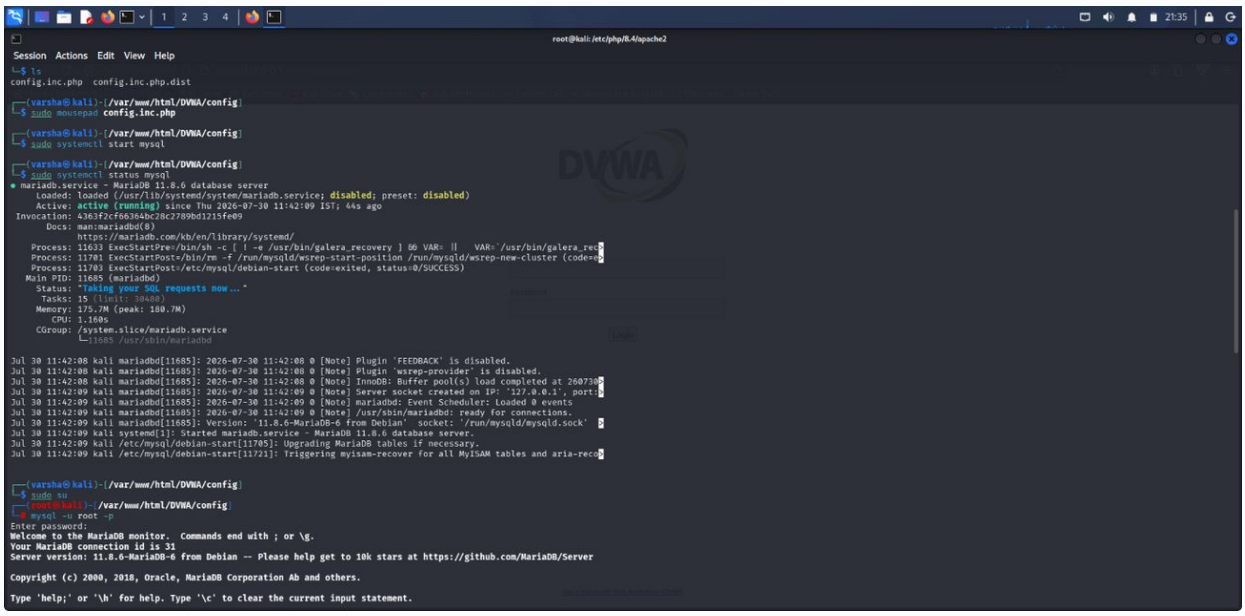


Figure 23. Starting MariaDB and logging in to the console

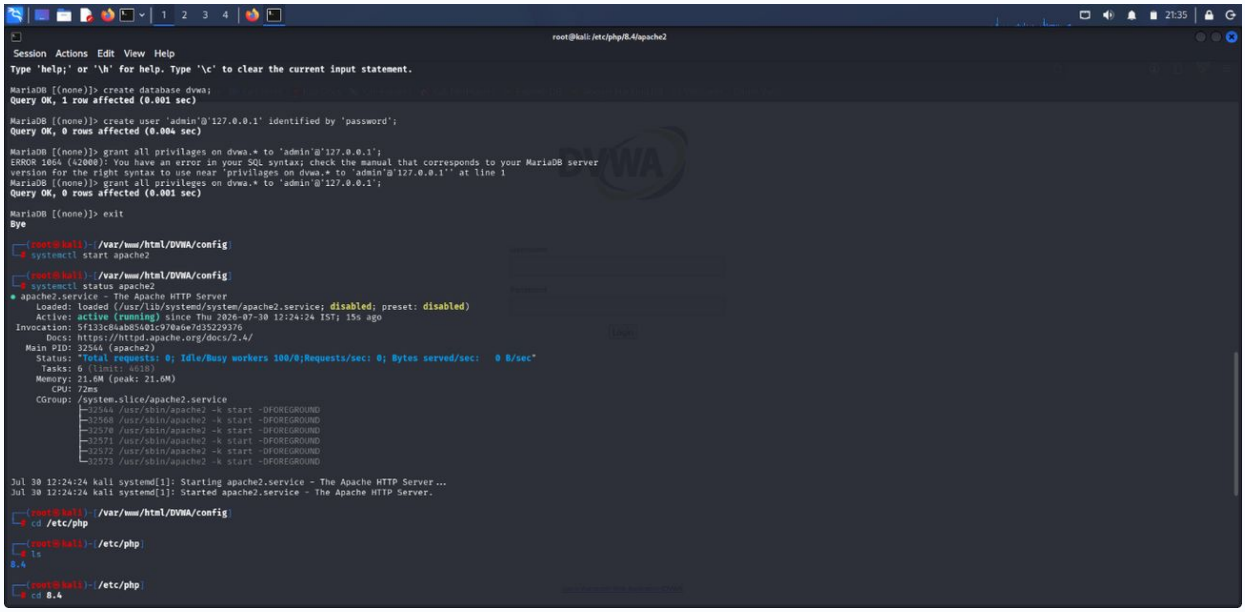


Figure 24. Creating the dvwa database and user

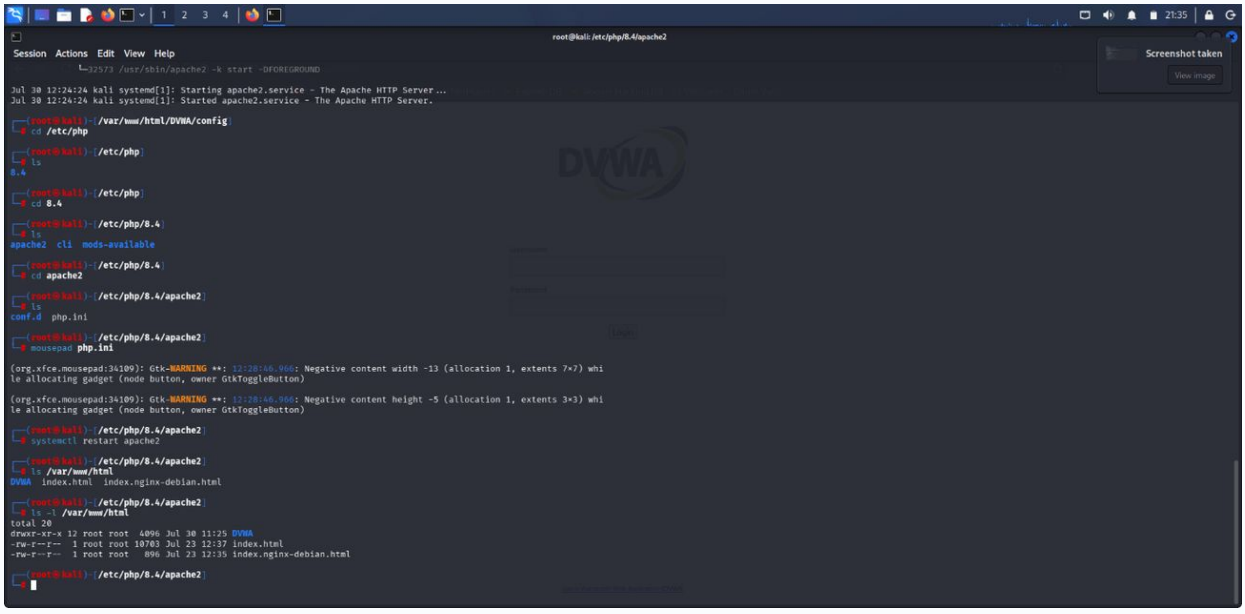


Figure 25. Starting Apache and configuring PHP

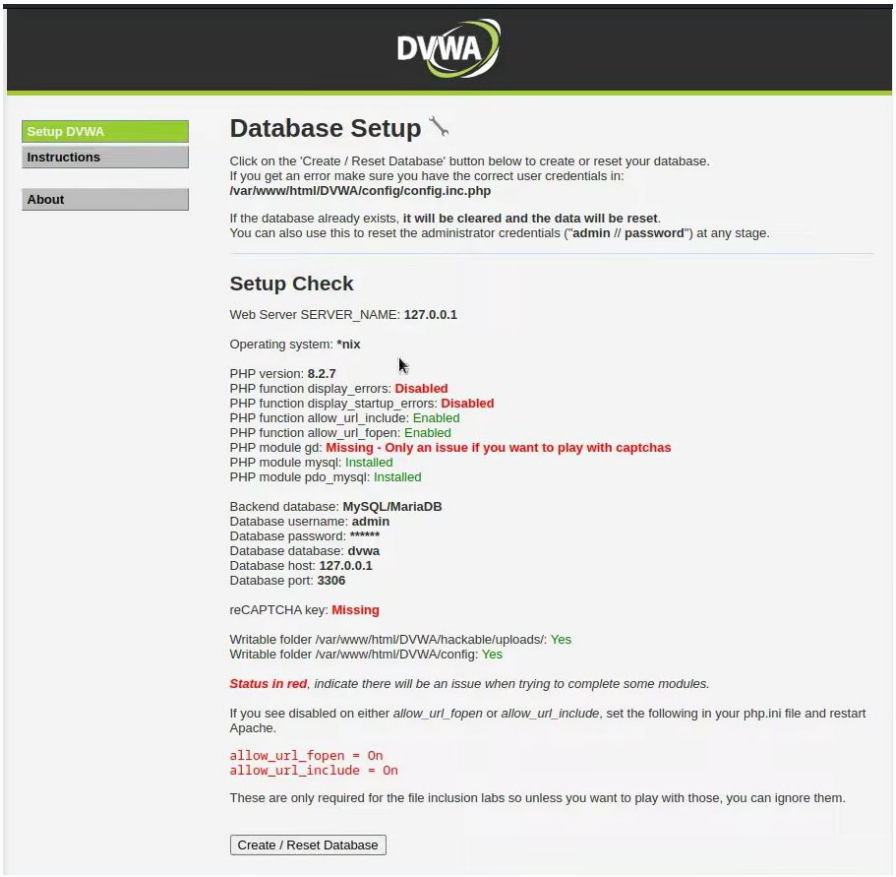


Figure 26. DVWA setup page: configuration checks before database initialization

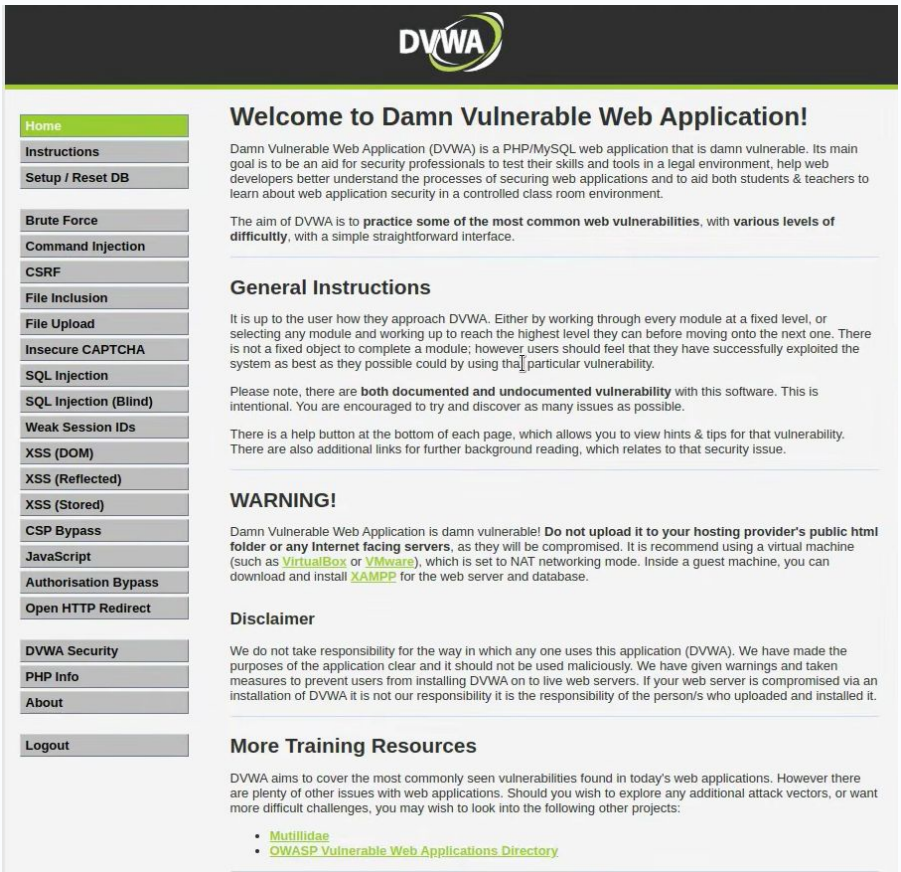


Figure 27. DVWA home page after successful installation

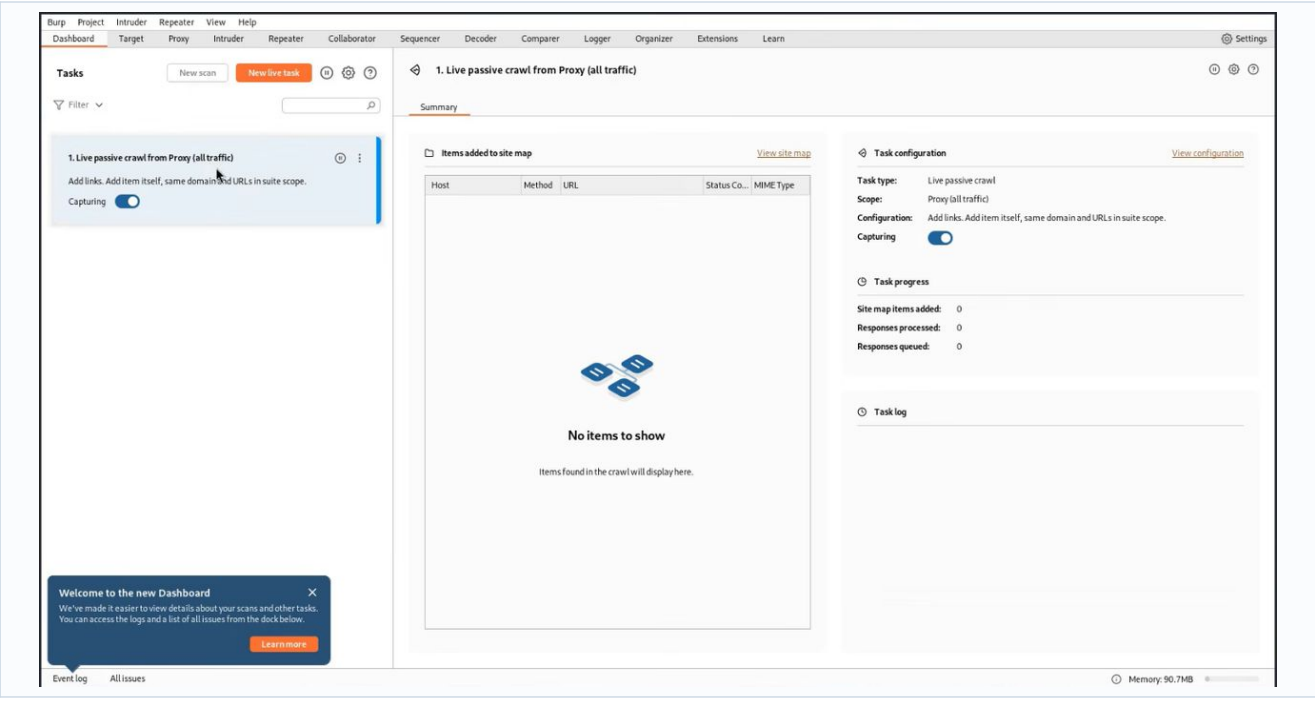


Figure 28. Burp Suite started for the brute force exercise

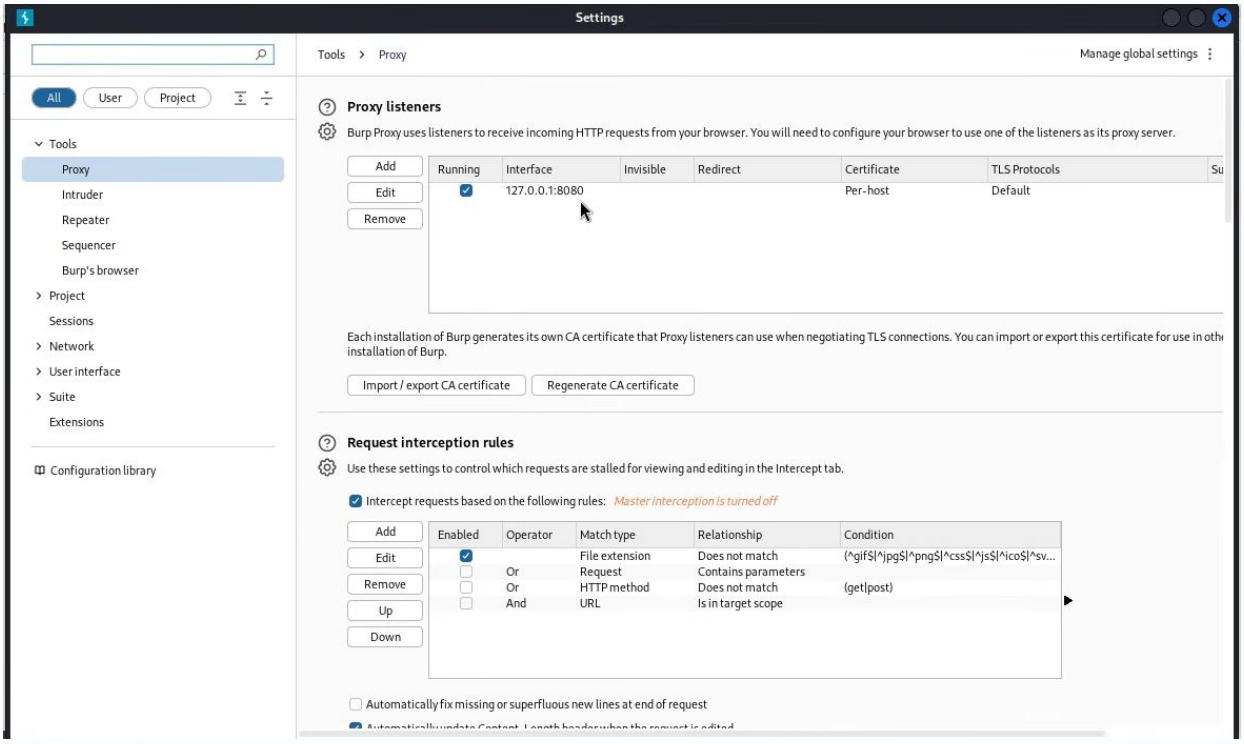


Figure 29. Proxy setup: FoxyProxy and Burp Intercept

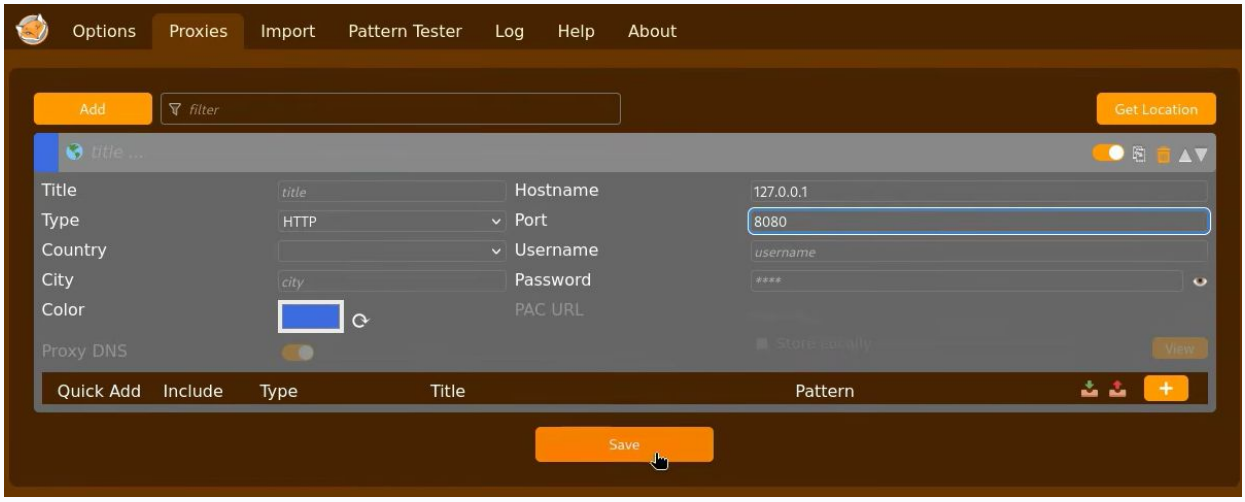


Figure 30. Proxy setup (continued)

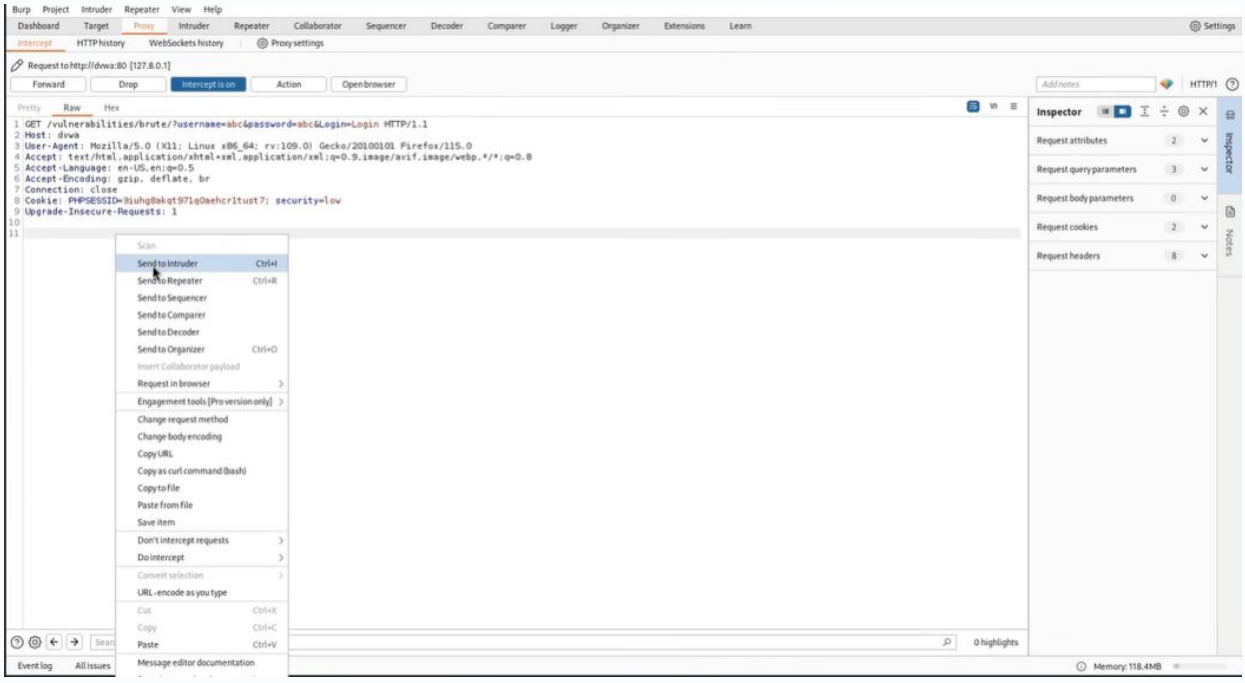


Figure 31. Sending the captured login request to Intruder

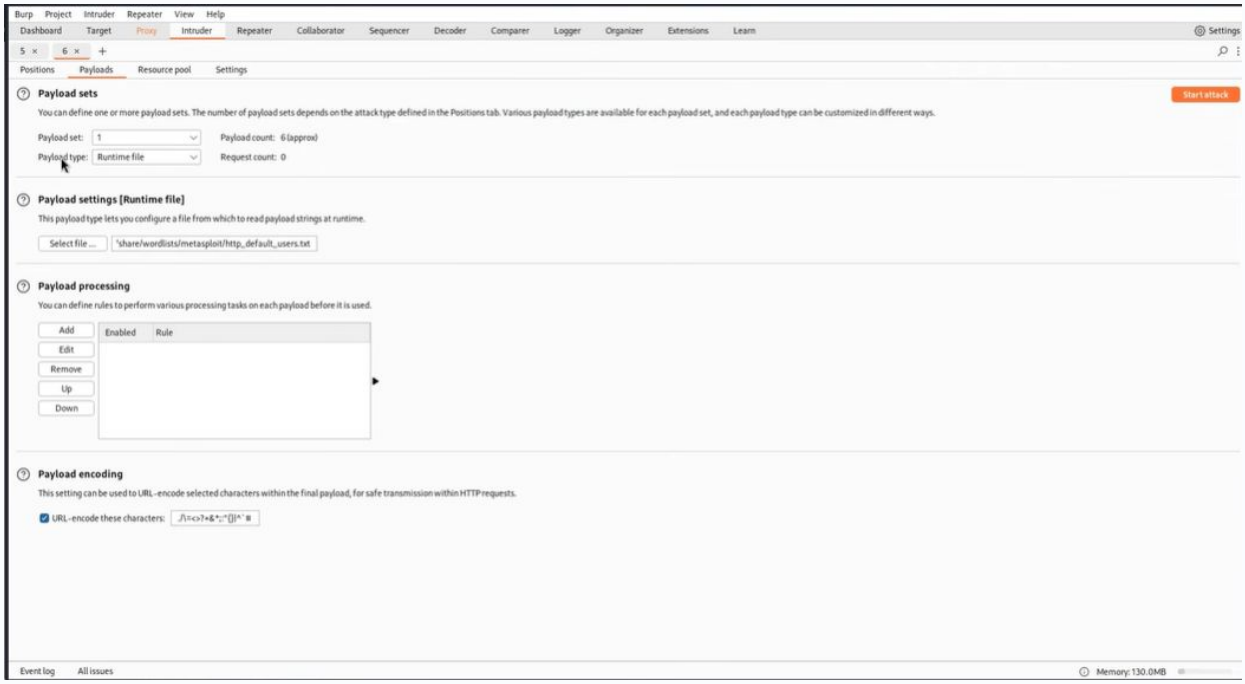


Figure 32. Intruder payload set 1: usernames (wordlist)

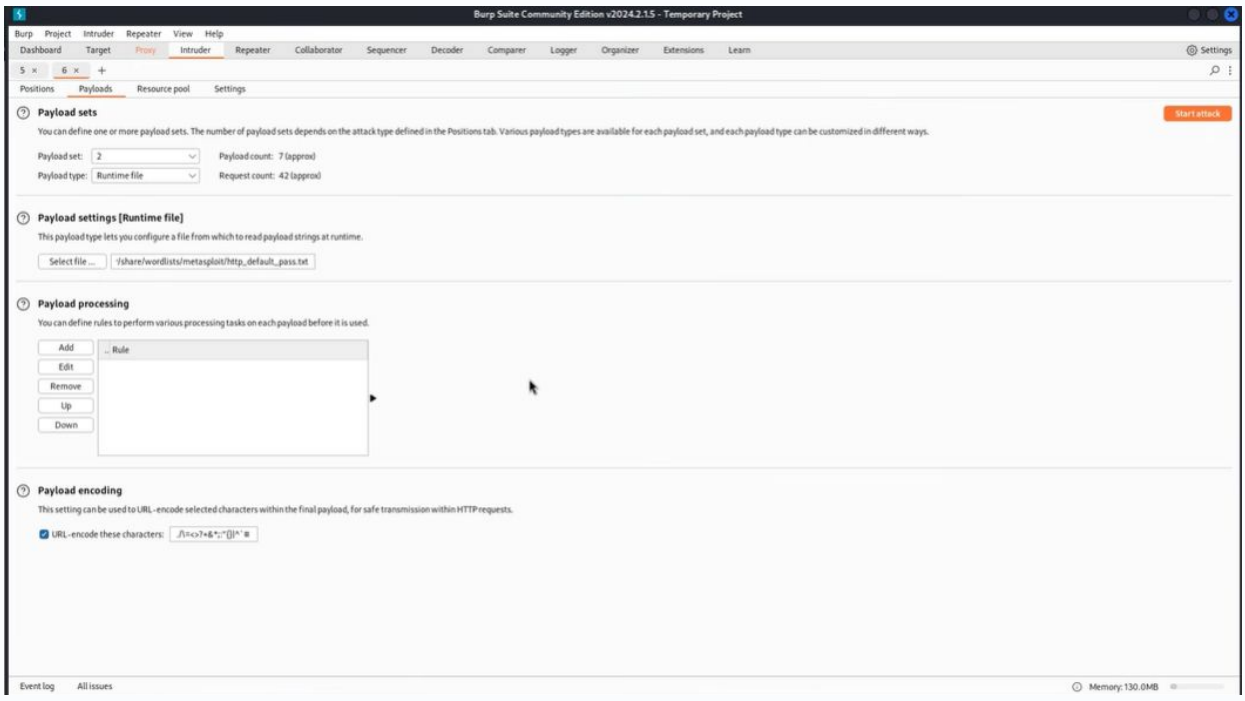


Figure 33. Intruder payload set 2: passwords (wordlist)

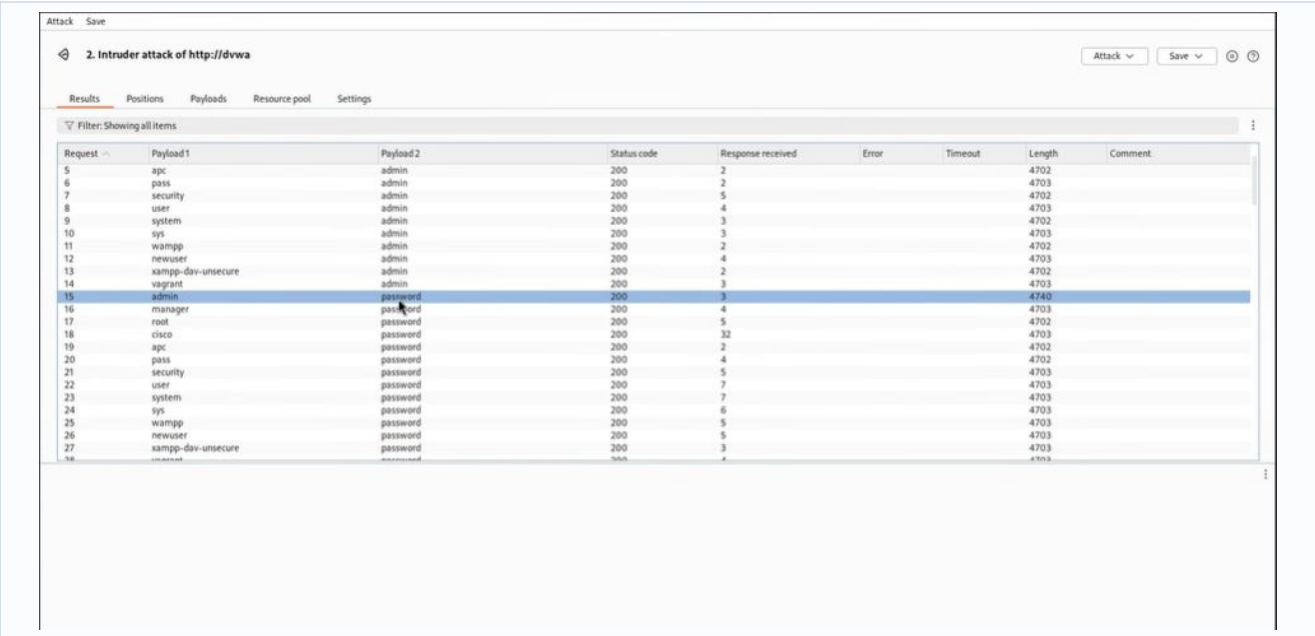


Figure 34. Intruder results: response-length outlier marks the valid login

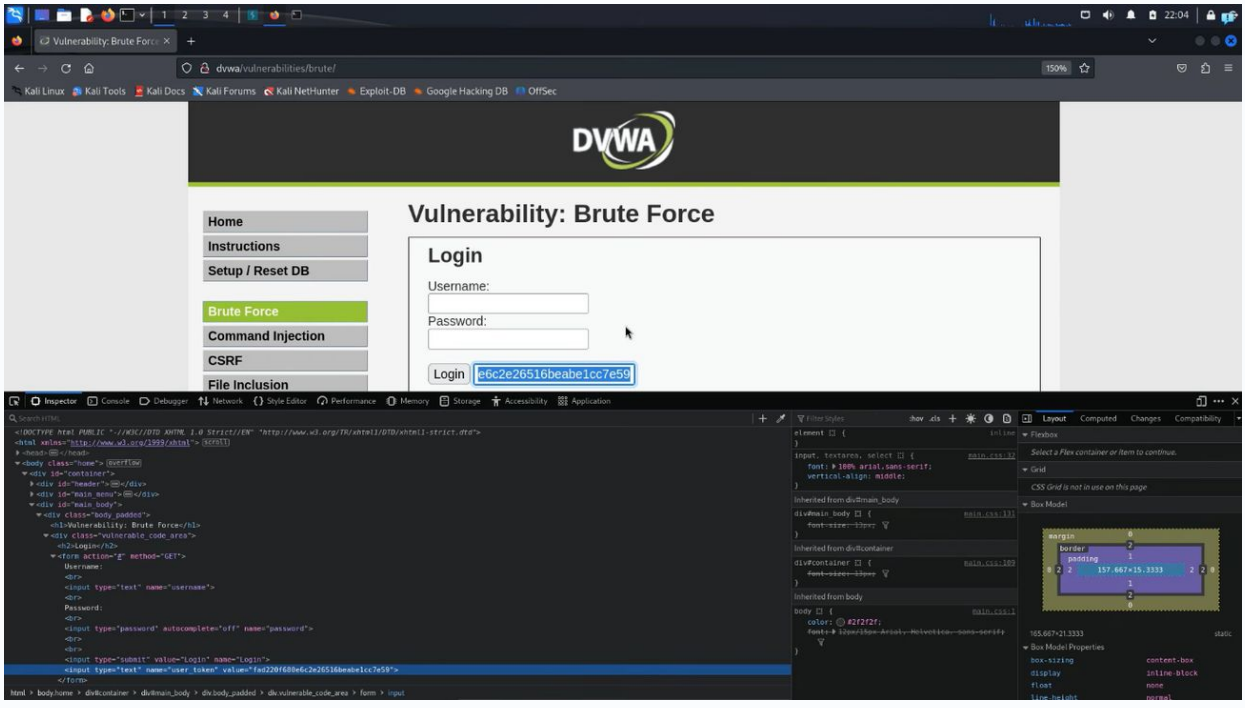


Figure 35. High level: anti-CSRF token revealed in the hidden form field

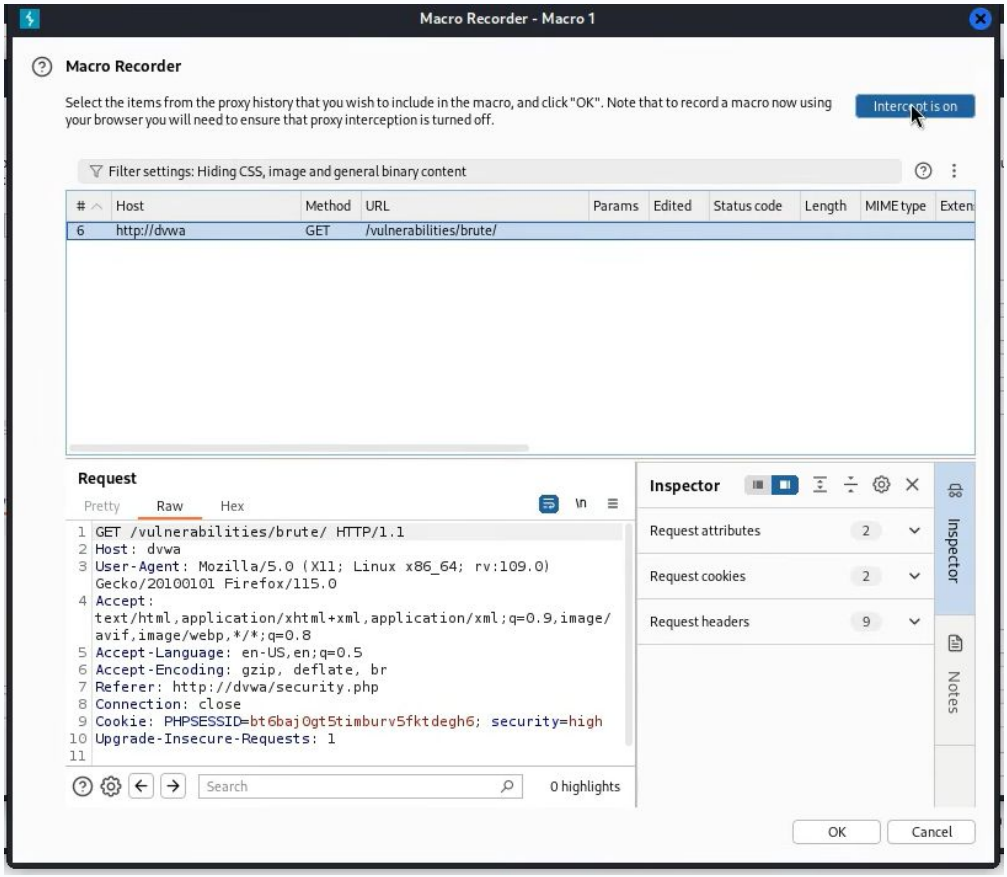


Figure 36. High level: Burp macro and session-handling rule to refresh the token

7. DVWA: command injection

Date: 31 Jul 2026

Level	Behavior	Payload that worked	Why
Low	No filtering	`127.0.0.1; ls`, `; whoami`, `; hostname`	User input is passed straight to the OS ping command
Medium	Blocks `;` and `&&`	`127.0.0.1 & ls`	Filter list is incomplete, single `&` still chains commands
High	Blocks more characters	`127.0.0.1&ls` (no space)	Filter strips a pattern that leaves a gap an attacker can use
Impossible	Strict validation	Rejected	Input is checked as a valid IP address before use

Fix: Never build shell commands from user input. Validate against a strict allowlist (here, a real IP format) and prefer APIs that avoid the shell.

Evidence from the lab

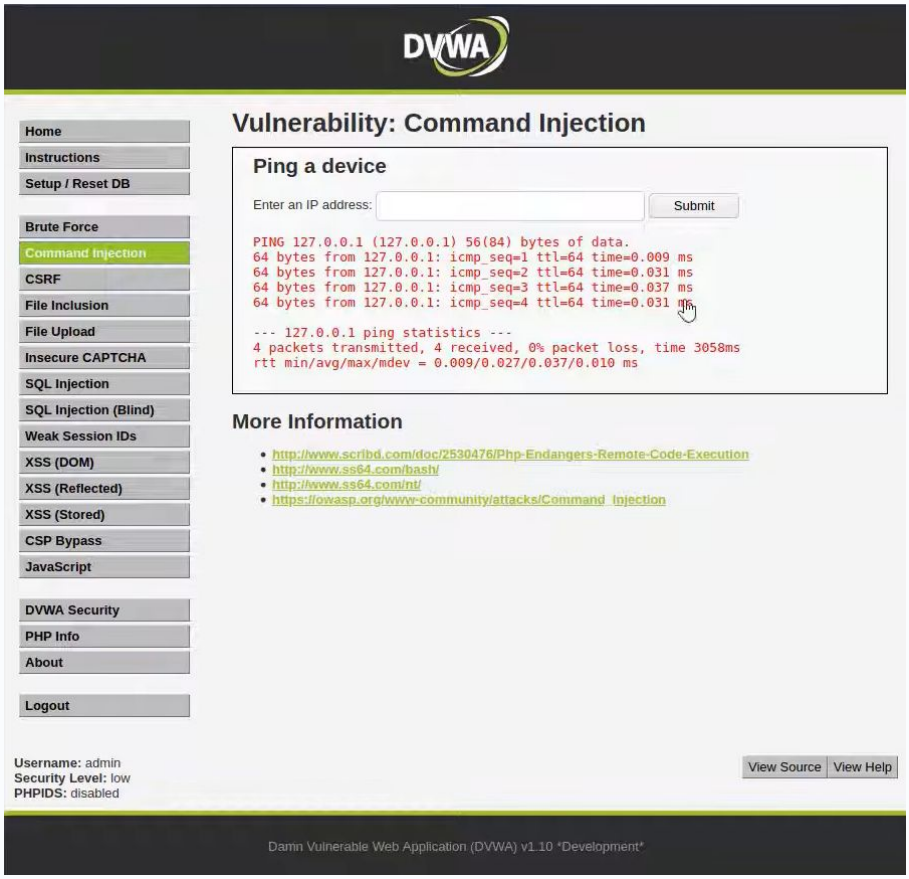


Figure 37. Low: normal ping request

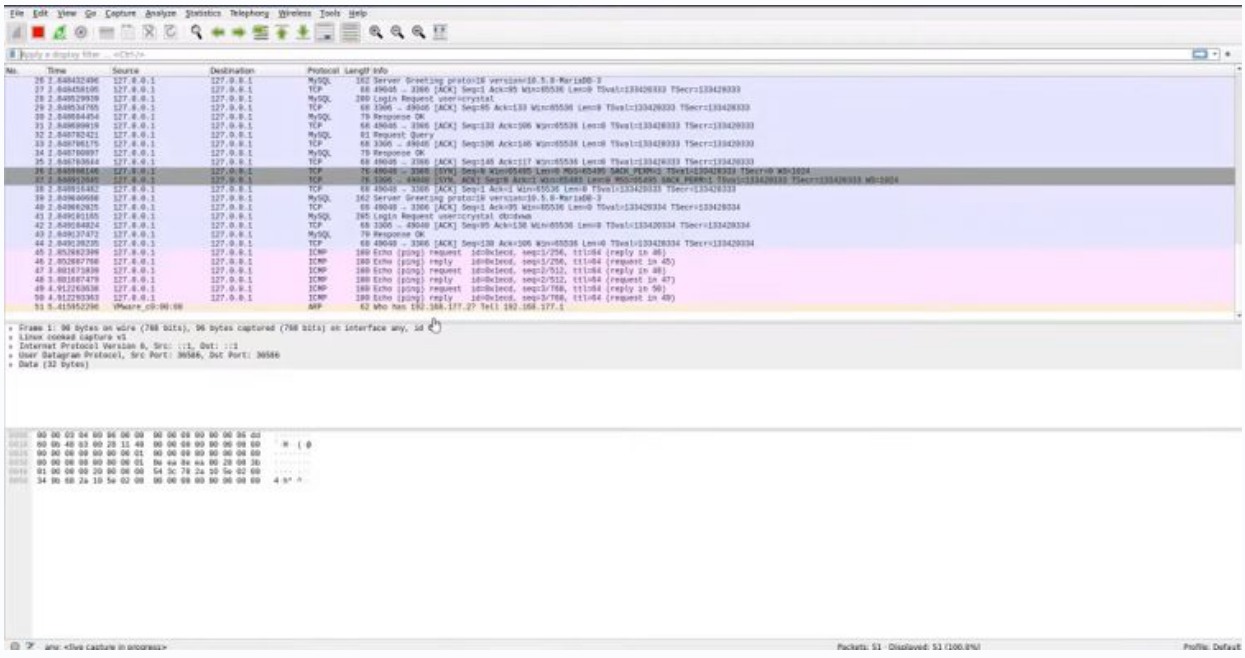


Figure 38. Low: ping output displayed

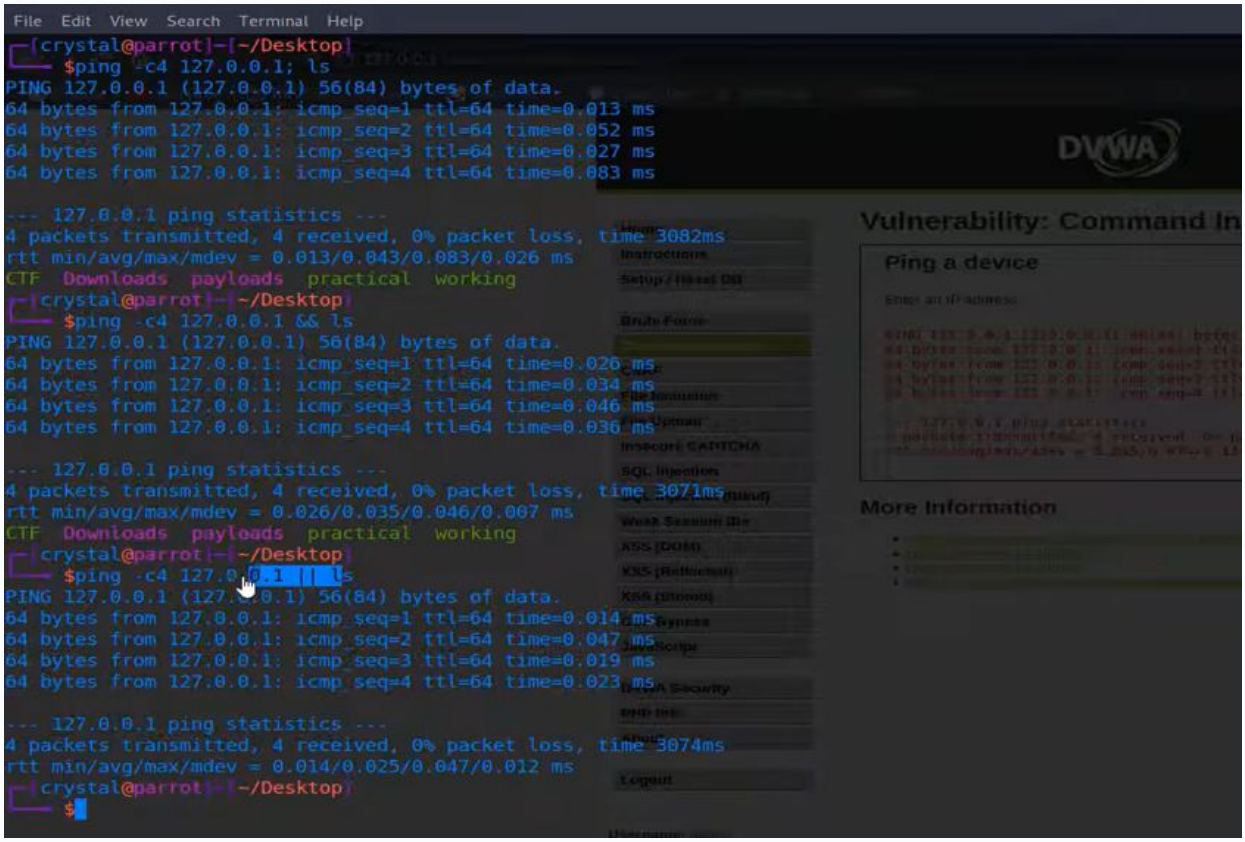


Figure 39. Low: injected OS commands executed (ls, whoami, hostname)

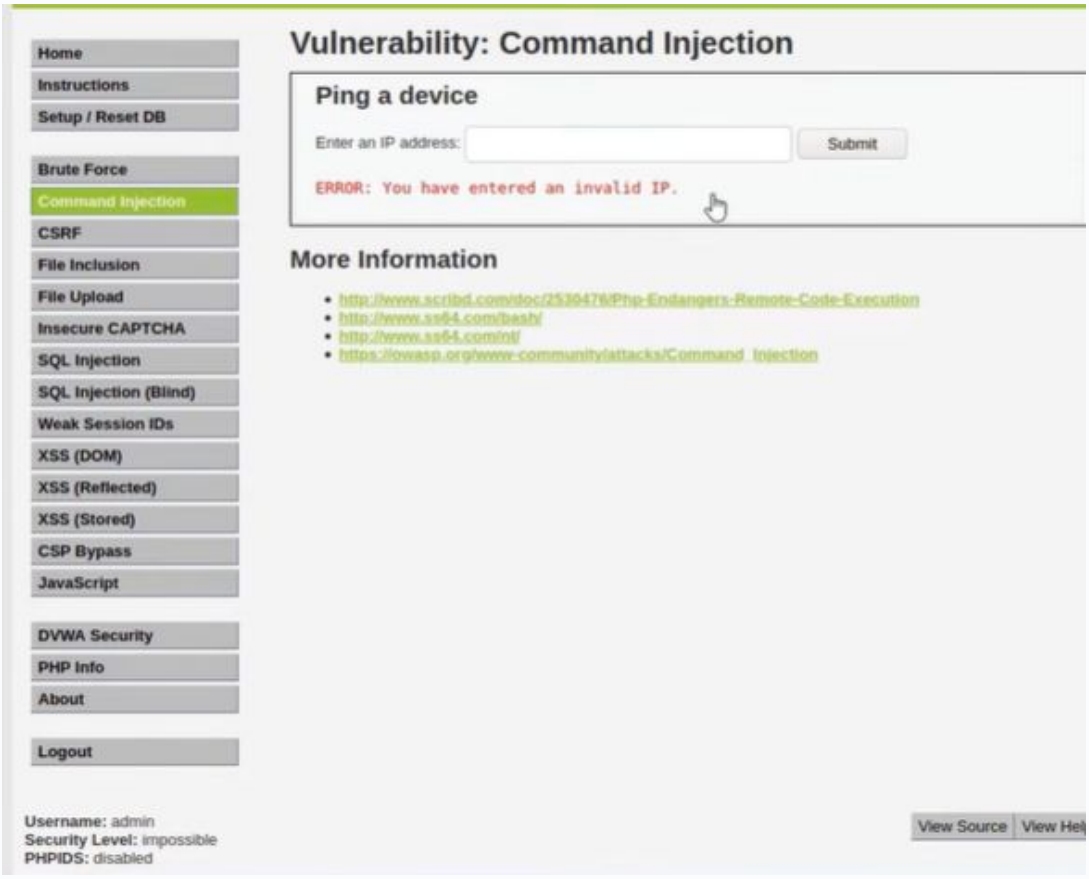


Figure 40. Impossible: invalid input rejected by strict IP validation

8. DVWA: CSRF

Date: 3 Aug 2026

- **Low:** Captured the password-change request in Burp, generated a proof-of-concept HTML page, and opened it while logged in. The browser sent the request automatically and the password changed. No CSRF protection exists.
- **Medium:** Additional request validation (Referer-header checking, depending on DVWA version) gives partial protection only.
- **High:** The form contains a per-page **CSRF token**. Replaying the old request or the old PoC failed because the token was invalid.
- **Impossible:** Strong token validation plus further checks; the PoC was rejected.

Fix: Anti-CSRF tokens bound to the session, `SameSite` cookies, and re-authentication for sensitive actions.

Evidence from the lab

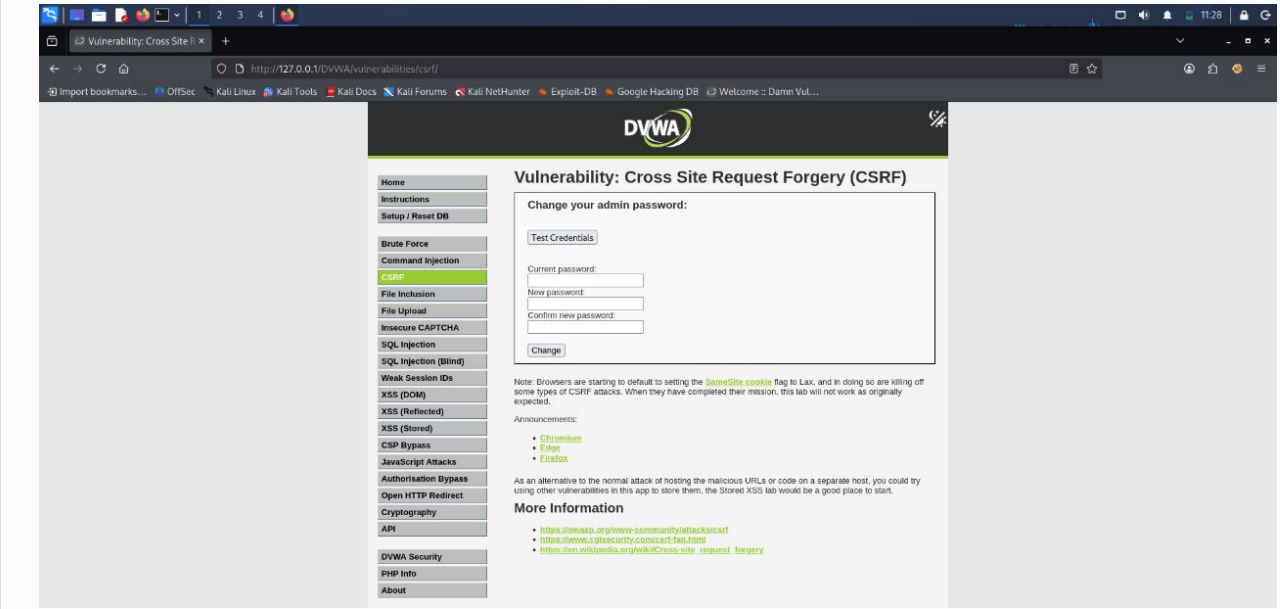


Figure 41. CSRF module in DVWA (Low)

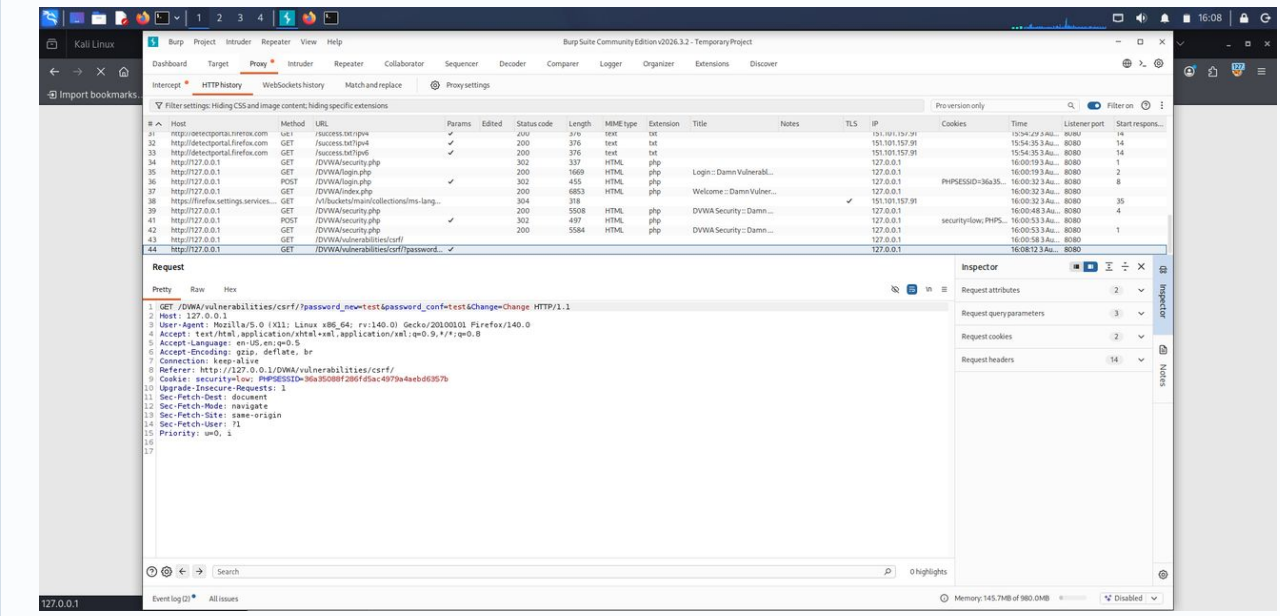


Figure 42. Password-change request captured in Burp

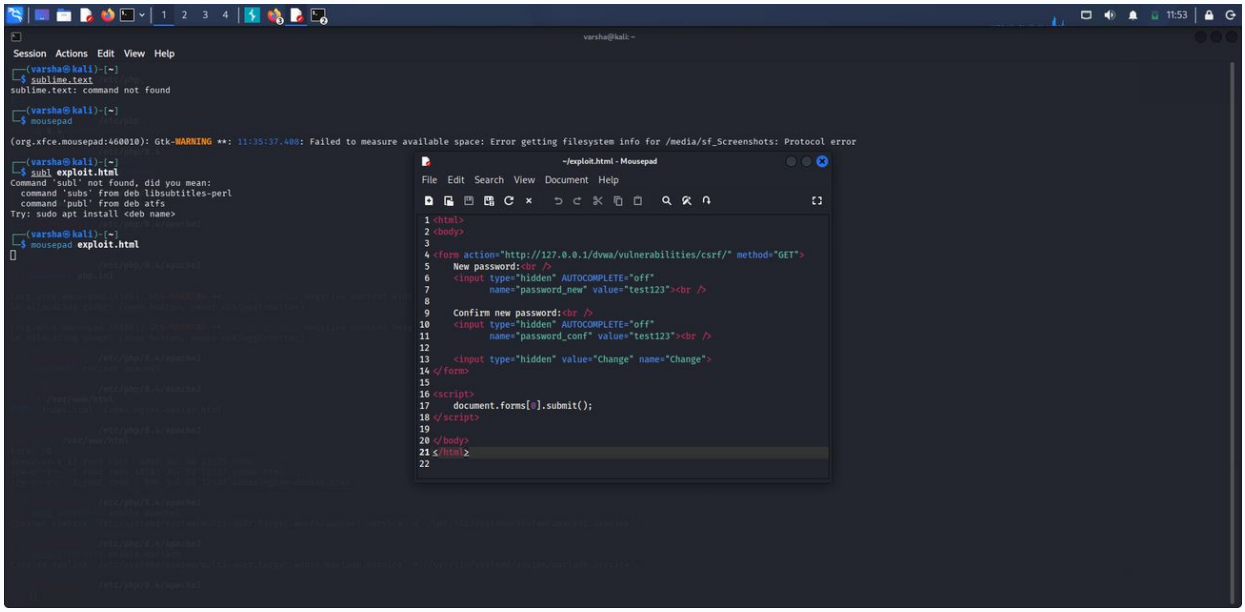


Figure 43. Proof-of-concept HTML generated from the request

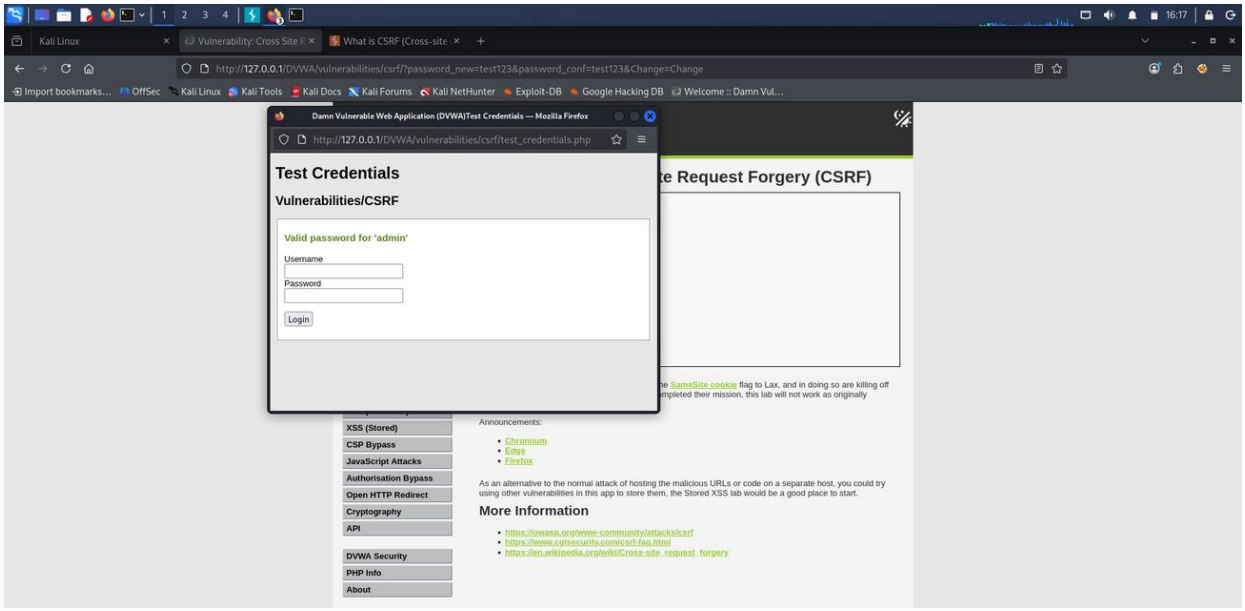


Figure 44. Password changed by the proof of concept without using the form

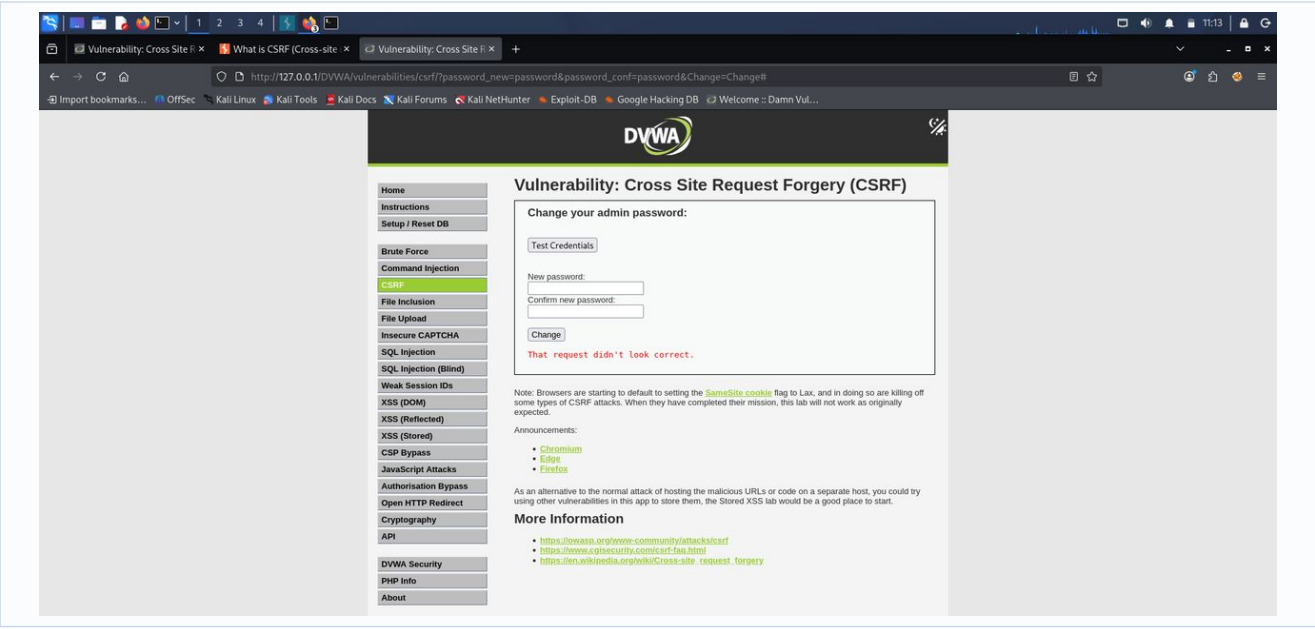


Figure 45. High: replayed request fails because the CSRF token is invalid

9. DVWA: file inclusion (LFI / RFI)

Date: 4 Aug 2026 · **Level:** Low

- **Normal use:** `?page=file1.php` loads the file named in the URL, with no validation.
- **Local File Inclusion:** `?page=/etc/passwd` and `?page=../../../../../../etc/passwd` displayed the system's `/etc/passwd`, showing both direct paths and directory traversal work.
- **Remote inclusion behavior:** an external URL in the `page` parameter was accepted and loaded inside the application.

Fix: Never pass user input to include functions. Use an allowlist of permitted files, disable remote includes (`allow_url_include=Off`), and normalize paths.

Evidence from the lab

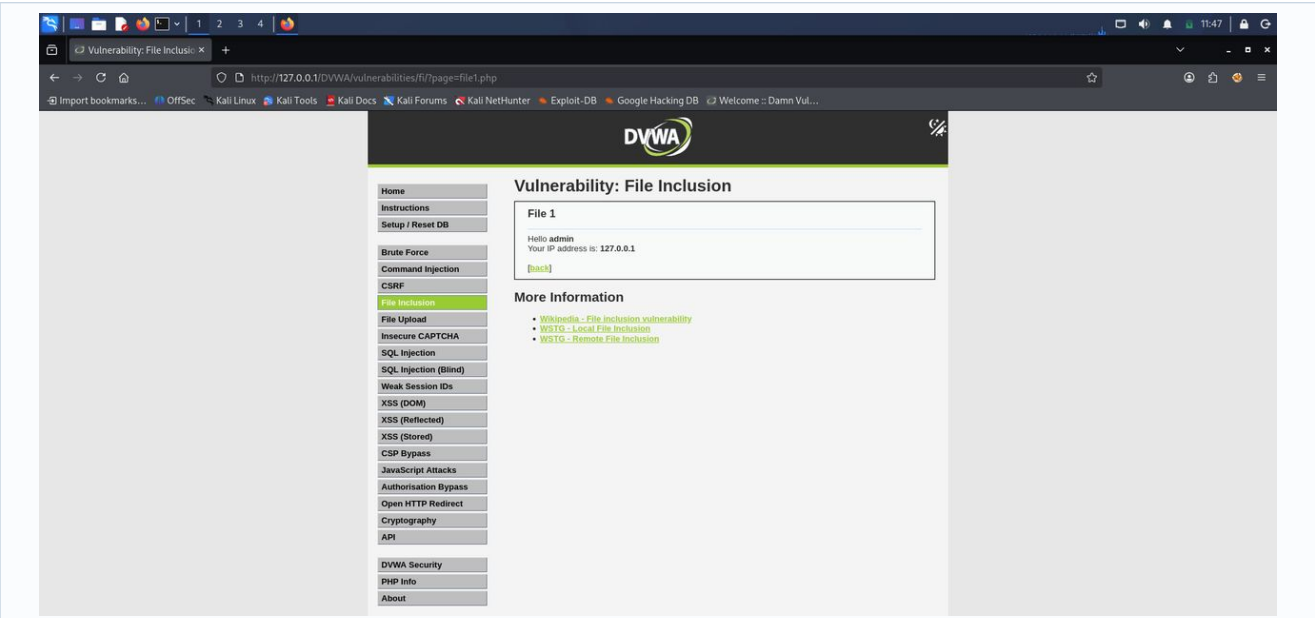


Figure 46. Normal include: ?page=file1.php

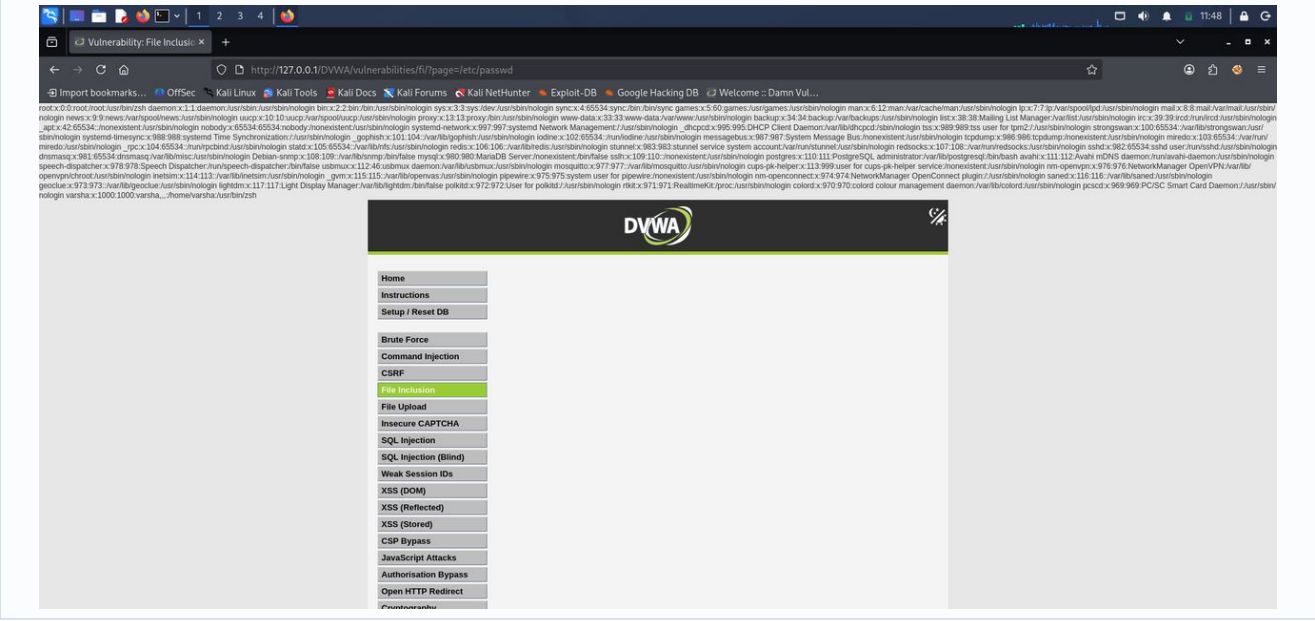


Figure 47. LFI: /etc/passwd displayed

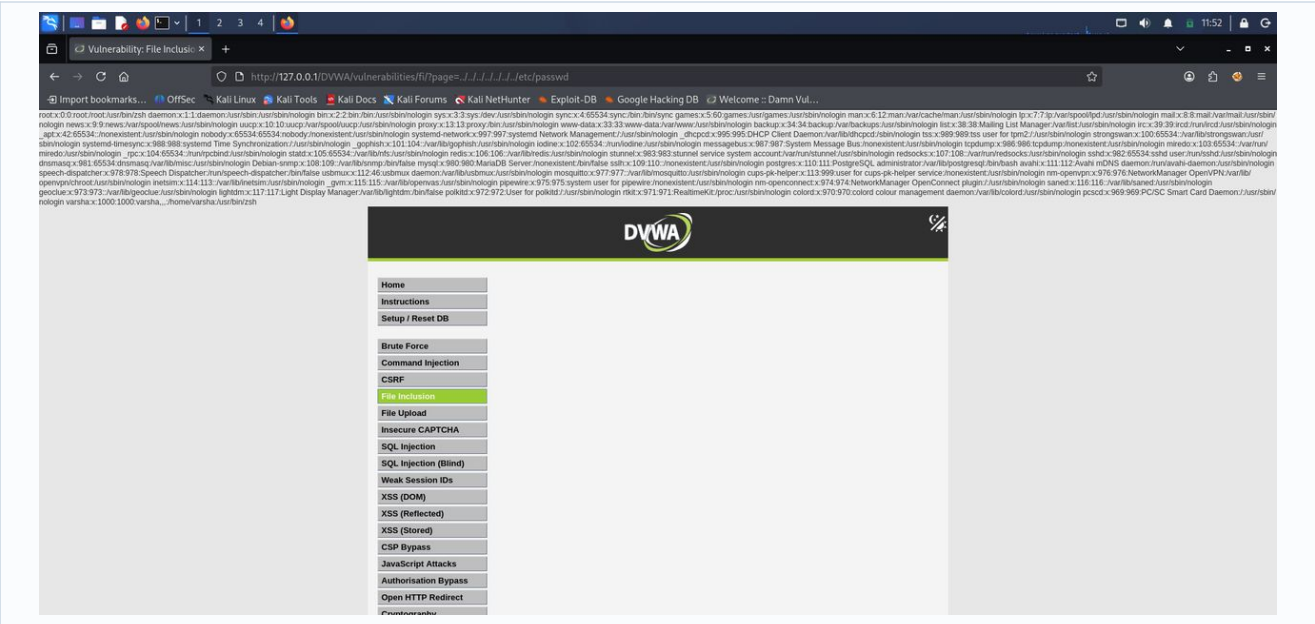


Figure 48. Directory traversal with ../ sequences

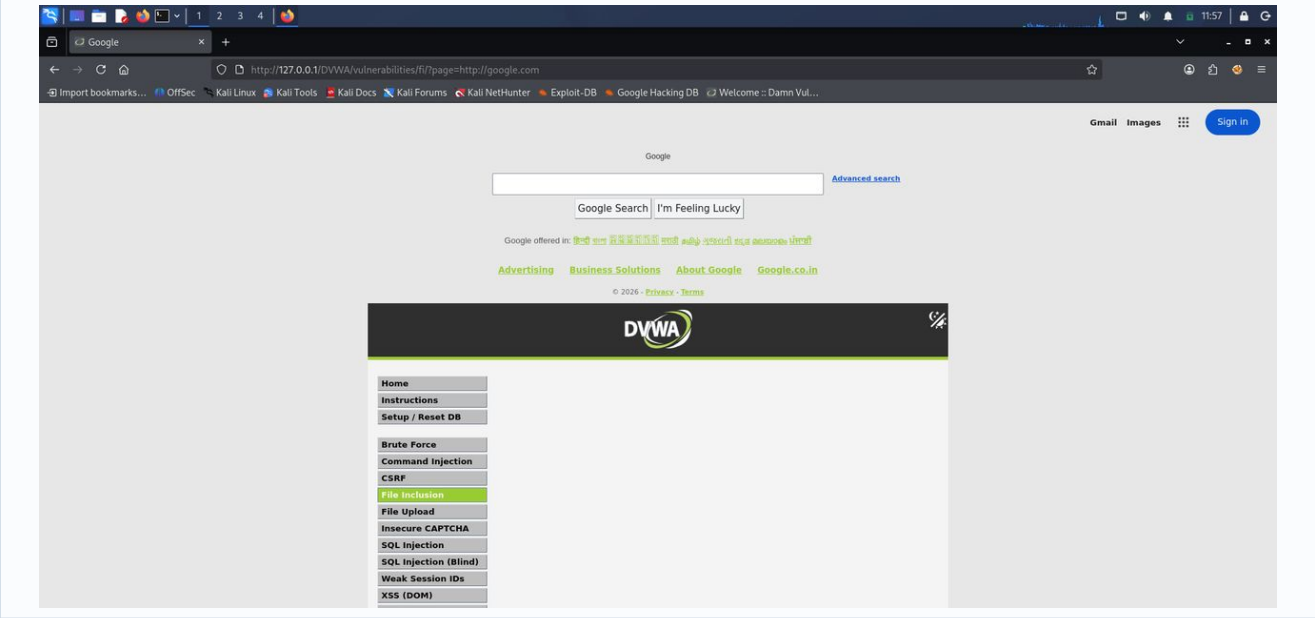


Figure 49. External URL accepted by the page parameter

10. DVWA: unrestricted file upload

Date: 5 Aug 2026 · **Level:** Low

Uploaded a legitimate `.jpg` and the application accepted it. In the controlled DVWA environment I then uploaded a test PHP file, which was also accepted and was executed by the server when opened through its upload path. This confirms an **unrestricted file upload** weakness at Low security.

Fix: Validate file type by content (not just extension or MIME header), rename uploads, store them outside the web root, disable script execution in the upload directory, and cap sizes.

Evidence from the lab

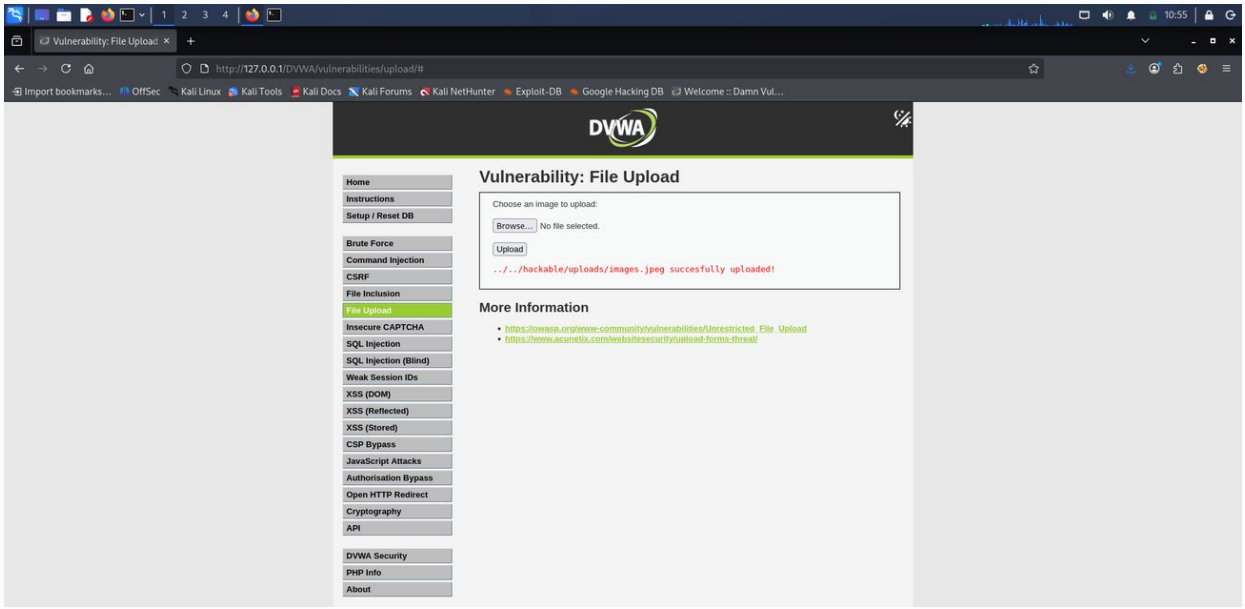


Figure 50. Legitimate image upload accepted

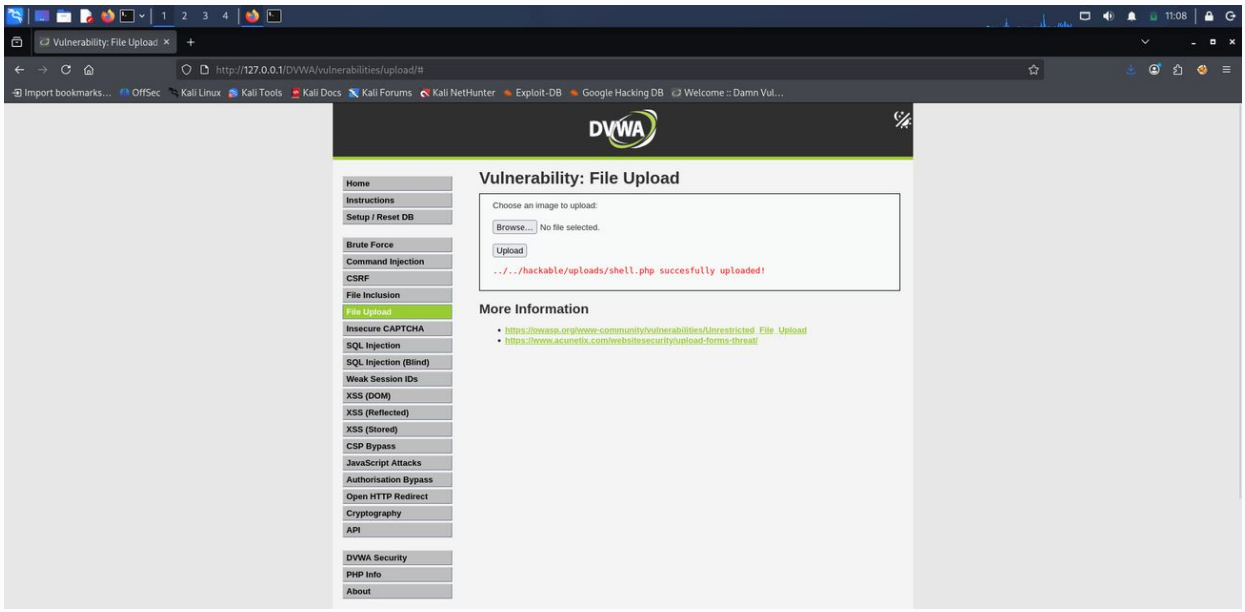


Figure 51. PHP test file accepted by the upload form

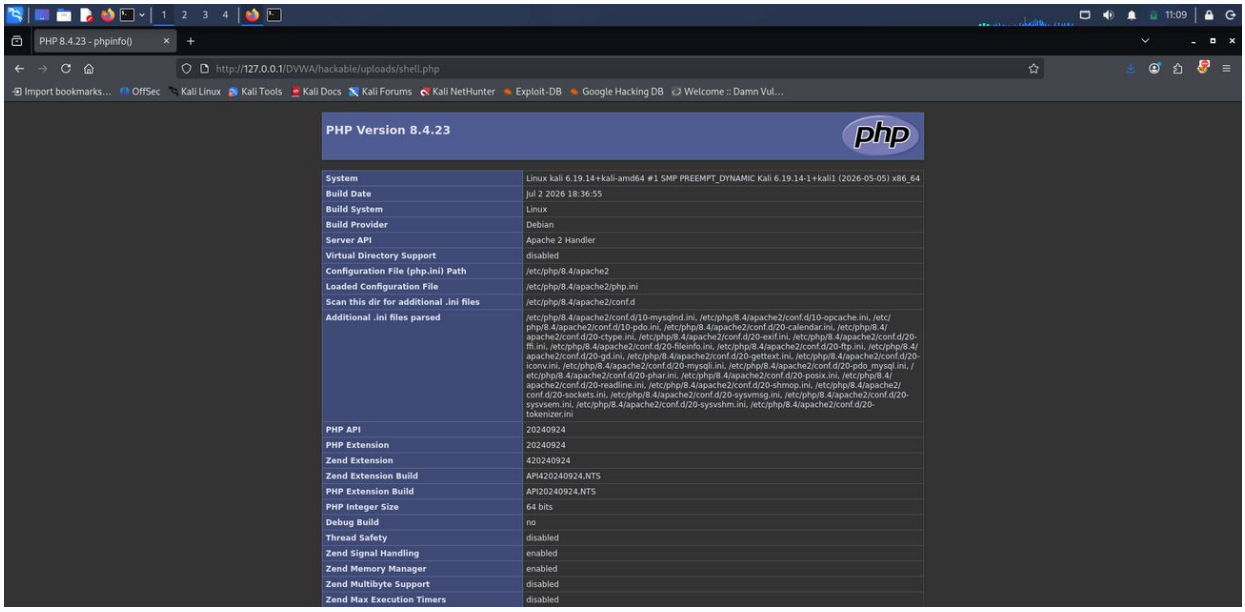


Figure 52. Uploaded file processed by the server

11. DVWA: reflected XSS

Date: 6 Aug 2026 · **Level:** Low

A JavaScript test payload entered in the input field was reflected straight back in the response and executed in the browser (alert dialog). Because the input is returned in the immediate response and not stored, this is **reflected XSS**.

Fix: Context-aware output encoding, input validation, a Content Security Policy, and `HttpOnly` cookies.

Evidence from the lab

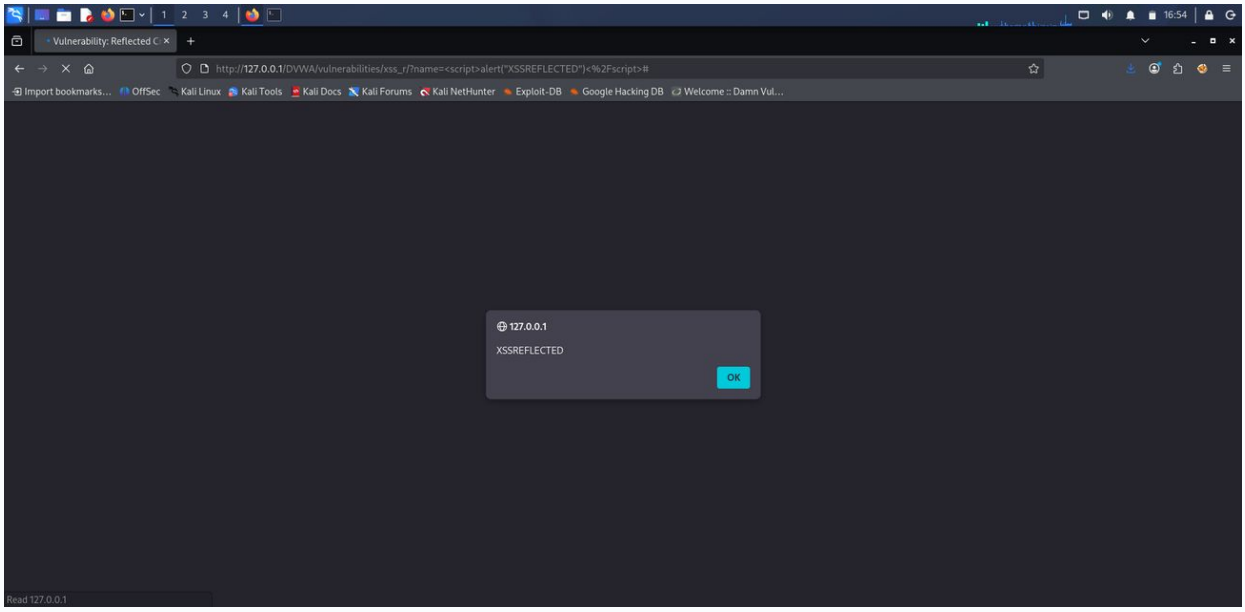


Figure 53. Reflected XSS: payload executes and shows an alert

12. DVWA: stored XSS

Date: 7 Aug 2026 · **Level:** Low

A test payload submitted through the guestbook's Message field was saved by the application and executed each time the page rendered. Because it persists on the server and affects every visitor, **stored XSS** is generally the more serious variant.

Evidence from the lab

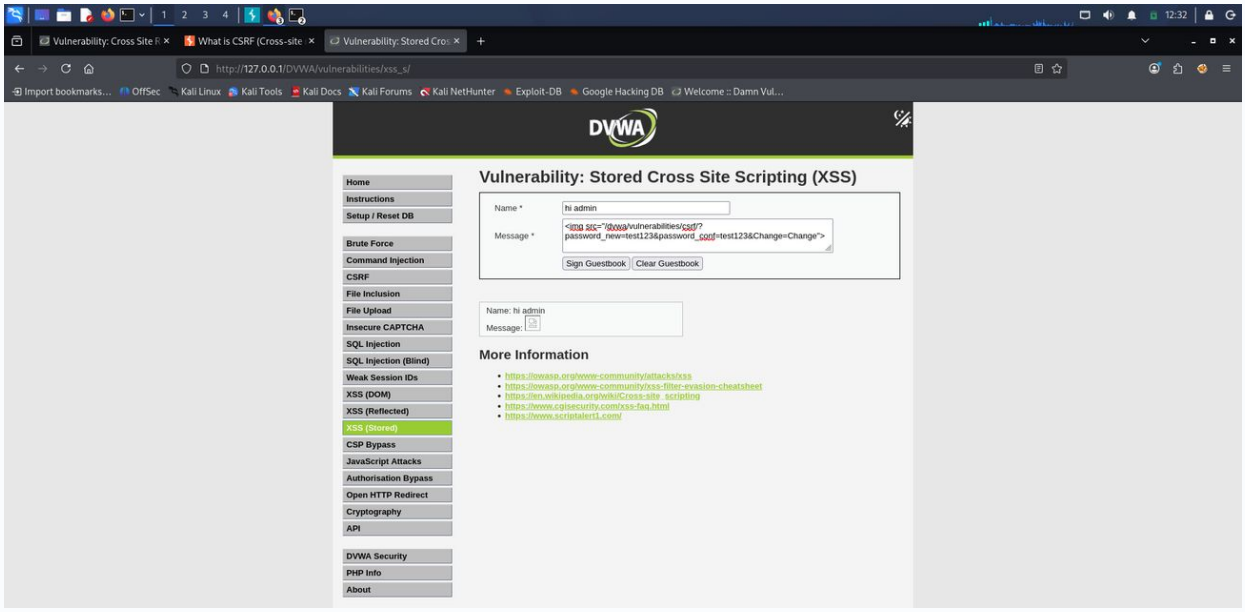


Figure 54. Stored XSS: payload submitted through the guestbook

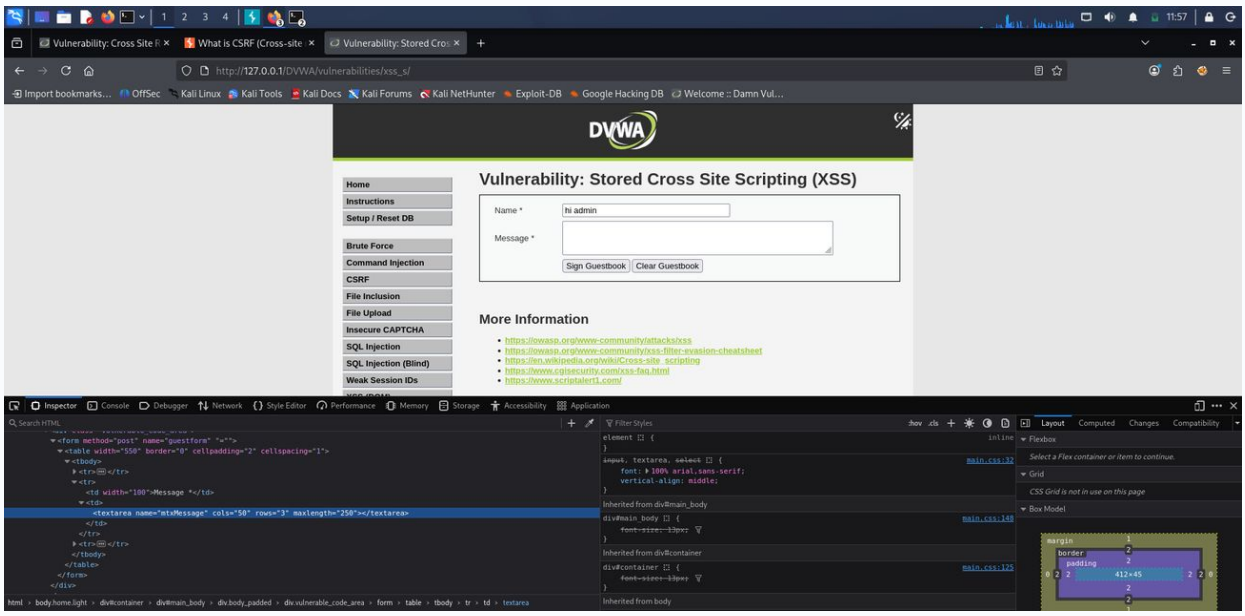


Figure 55. Stored XSS: payload rendered when the page loads

13. DVWA: information disclosure via phpinfo()

Date: 8 Aug 2026

An exposed `phpinfo()` page reveals the PHP version, OS, web server details, loaded modules, `php.ini` settings, upload limits, filesystem paths, and environment variables. Attackers use that to match known vulnerabilities to the exact version and to see whether risky features (uploads, remote includes) are enabled.

Fix: Remove diagnostic pages from production and restrict them to trusted admin access.

Evidence from the lab



Figure 56. phpinfo() output exposing server configuration

14. Spoofing: concepts and lab demonstrations

Concepts covered: IP, ARP, DNS, email, website, MAC, caller ID, GPS and biometric spoofing, with typical effects (credential theft, traffic interception, phishing, MITM) and defenses (MFA, HTTPS, SPF/DKIM/DMARC, DNSSEC, Dynamic ARP Inspection, monitoring).

IP spoofing and decoy scanning (Nmap + Wireshark)

- `nmap -e eth0 -S <fake-ip> <target>` failed with a bind error, because the address was not configured on the local interface.
- Wireshark on `eth0` was used to watch the traffic.
- `nmap -D RND:10 <target>` (decoy scan) produced TCP SYN packets that appeared to come from multiple source addresses, showing how decoys can obscure a scan's true origin.

MAC address spoofing

- `macchanger -s eth0` showed the original MAC; `sudo macchanger -r eth0` assigned a random one, confirmed in the output.
- `sudo nmap --spoof-mac Apple <target>` made Nmap present a spoofed vendor MAC.

ARP spoofing / MITM (lab only)

Enabled IP forwarding on Kali, ran `arpspoof` against the Metasploitable lab VM and its gateway, and used `dsniff` to observe the intercepted traffic from the authorized target.

Defenses: static ARP entries or Dynamic ARP Inspection, encrypted protocols (HTTPS, SSH, VPN), and network monitoring for ARP anomalies.

Evidence from the lab

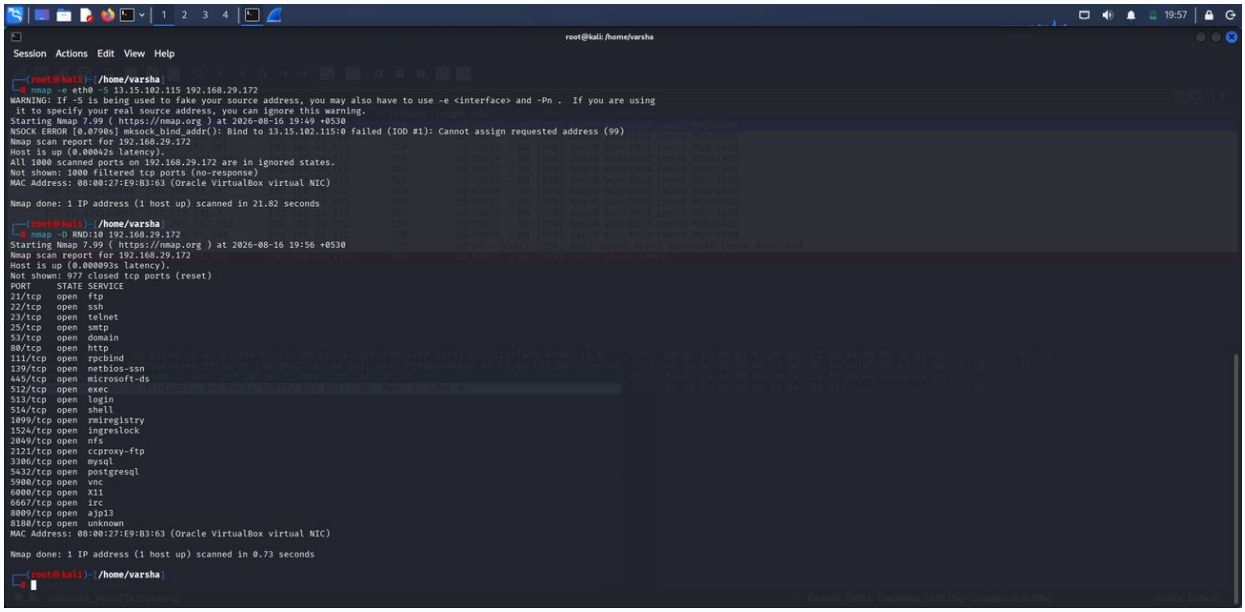


Figure 57. IP spoofing attempt with Nmap (-S) and the resulting bind error

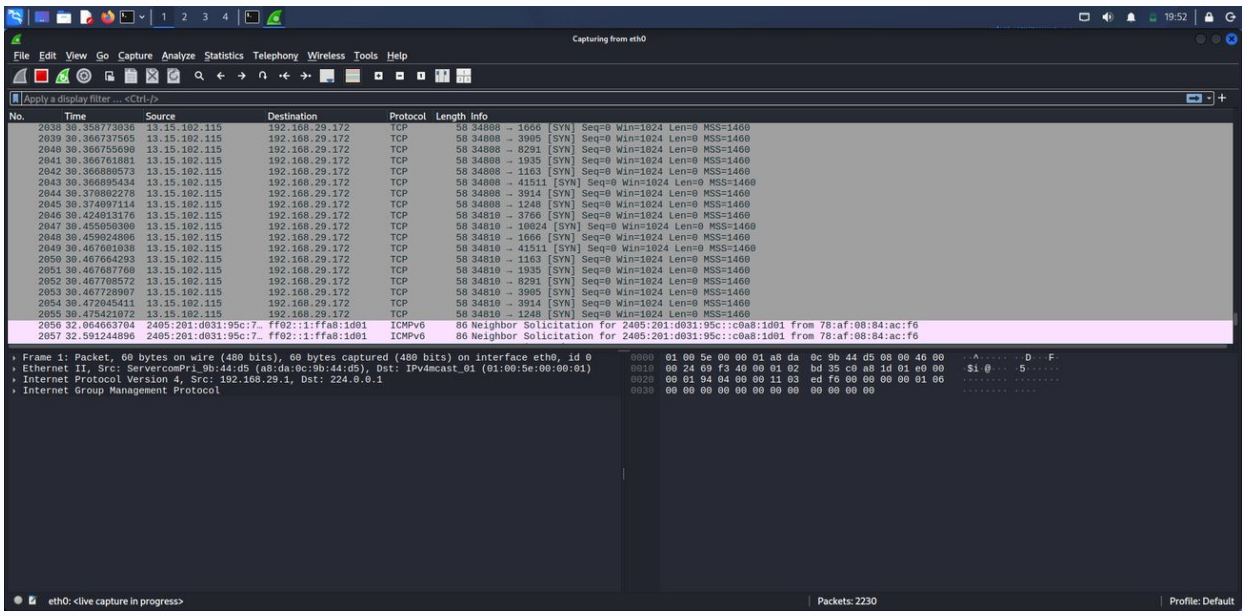


Figure 58. Wireshark capture while scanning

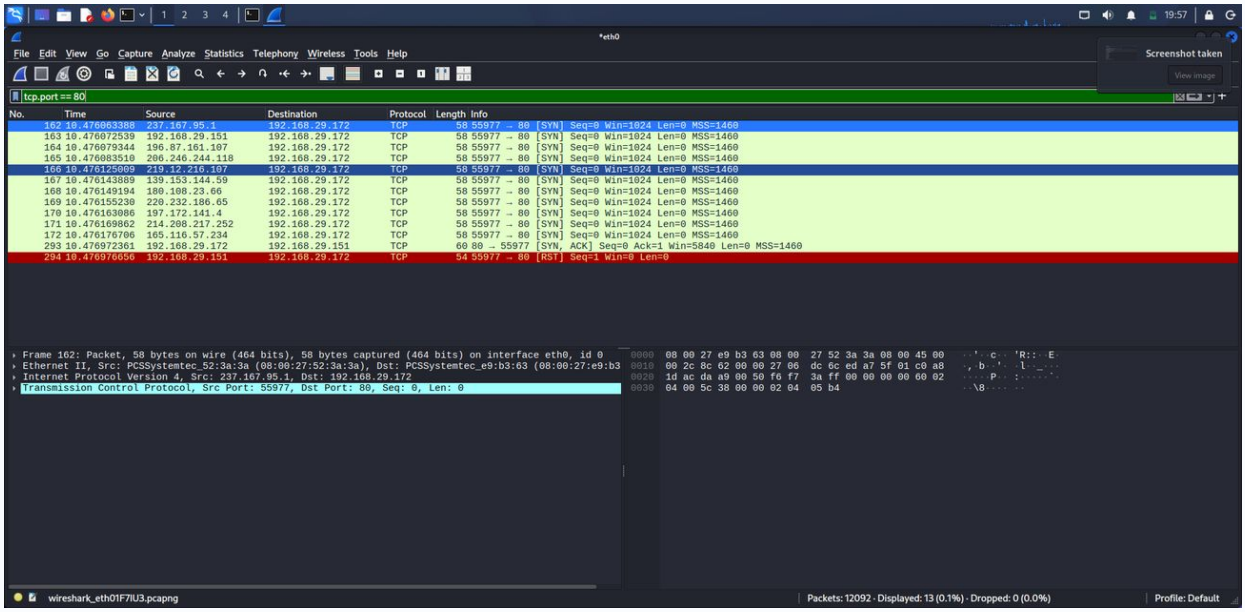


Figure 59. Decoy scan (-D RND:10): SYN packets from multiple apparent sources

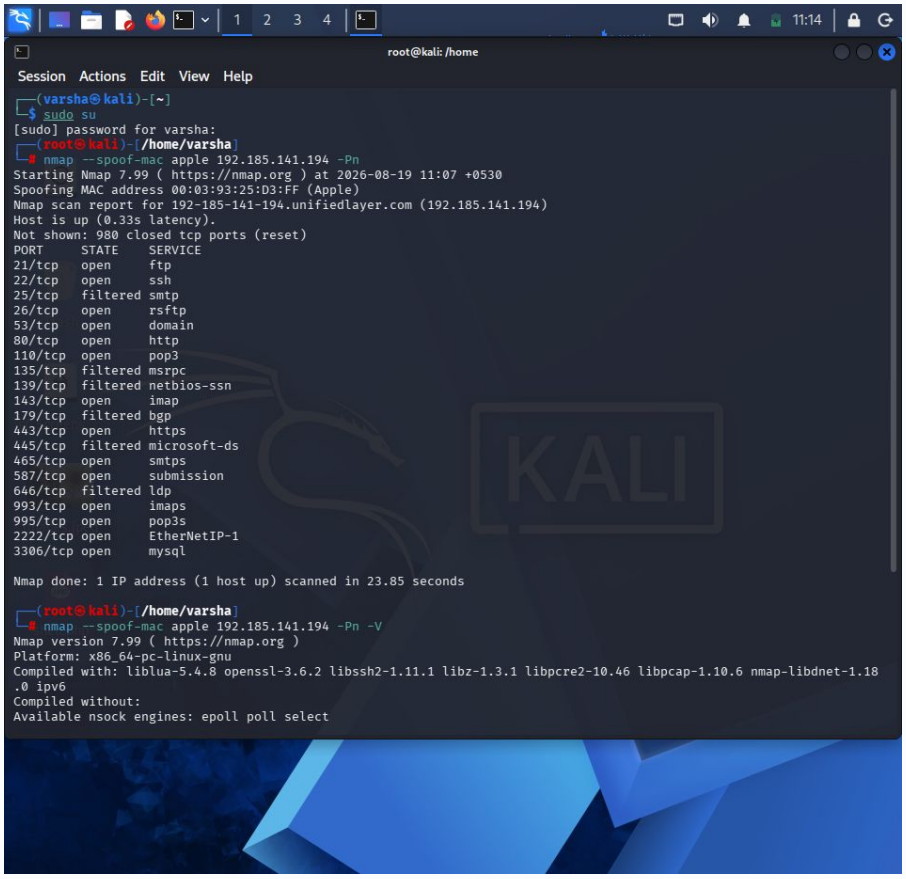


Figure 60. MAC address spoofing with macchanger (original vs. new MAC)

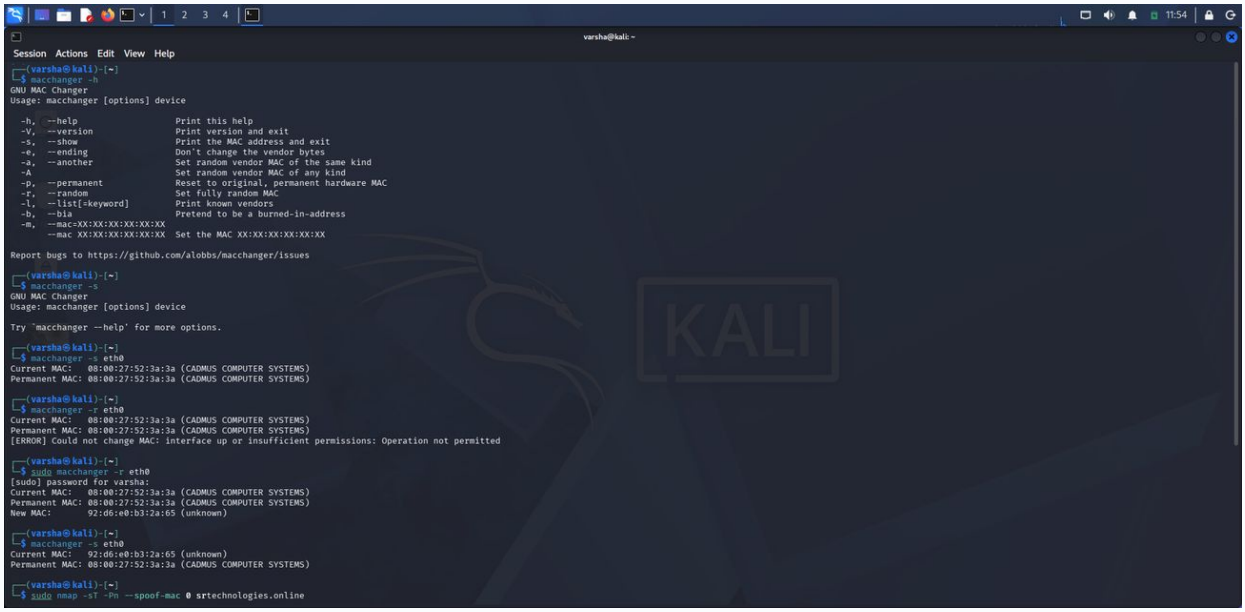


Figure 61. MAC spoofing with Nmap (--spoof-mac)



Figure 62. MAC spoofing evidence (continued)

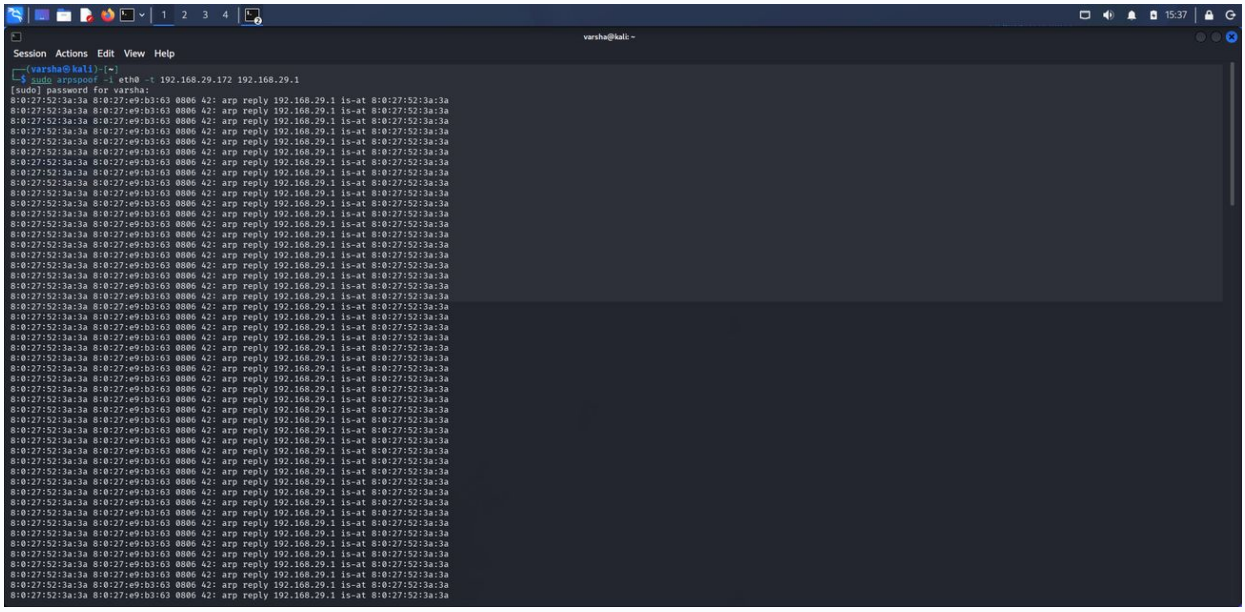


Figure 63. ARP spoofing setup against the lab target

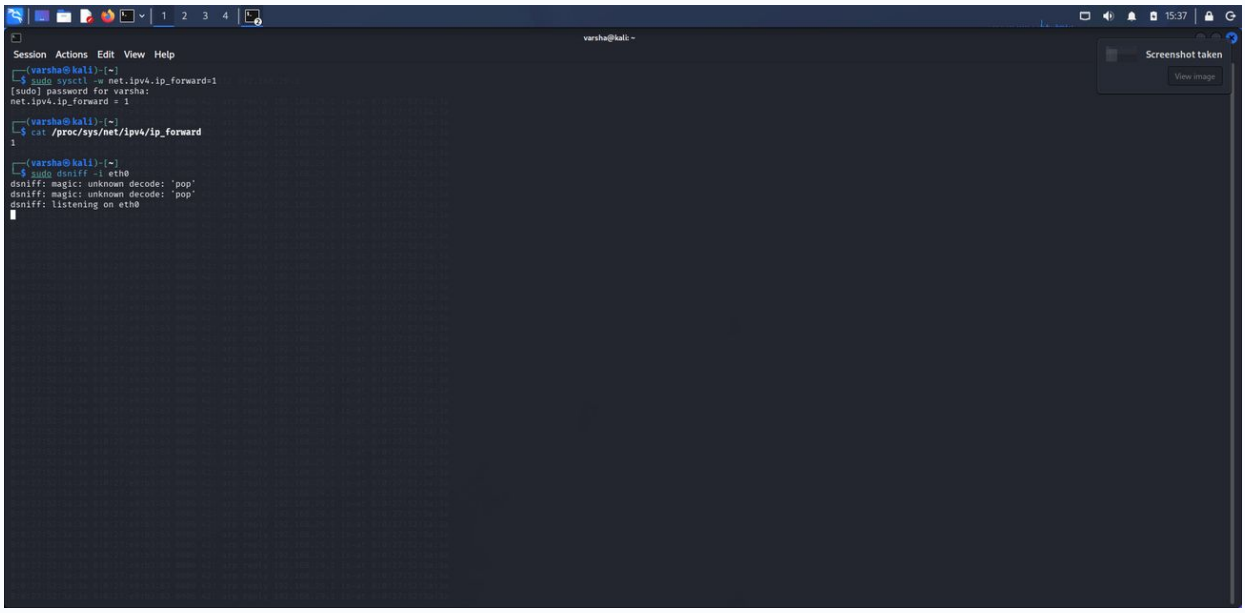


Figure 64. dsniff output from the authorized lab target

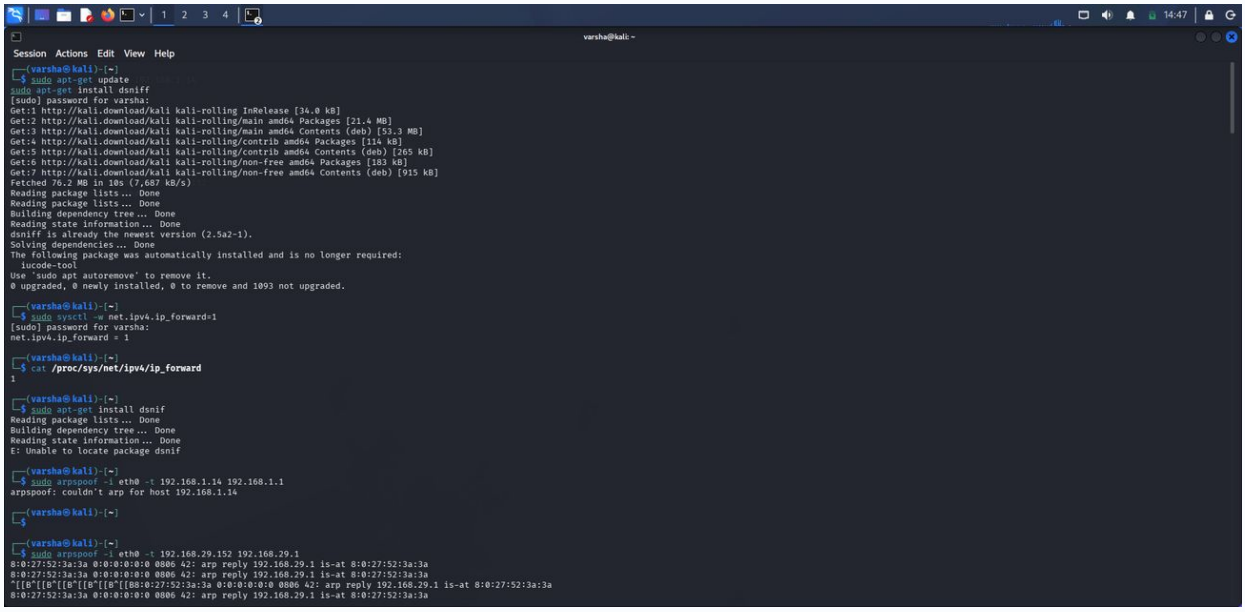


Figure 65. ARP spoofing evidence (continued)

15. Vulnerability scanning with OpenVAS / GVM

1. Installed OpenVAS (`apt install openvas`) and ran `gvm-setup`, which created the PostgreSQL database, GVM user, certificates, and feeds.
2. Verified the install with `gvm-check-setup` (final status: **GVM installation OK**).
3. Created an admin user, logged into the web interface at `https://127.0.0.1:9392`, and ran a **Task Wizard** scan against `127.0.0.1`.
4. **Report result:** 1 Medium, 3 Low, 30 Log findings, no Critical or High; overall CVSS **5.9 (Medium)**.
5. Used Nmap for discovery on my local network: `nmap -sn <subnet>/24` found **8 live hosts**, and `-oG - | grep Up | cut -d ' ' -f 2` extracted the IP list for follow-up.

The auto-generated admin password in the setup screenshot has been redacted.

Evidence from the lab

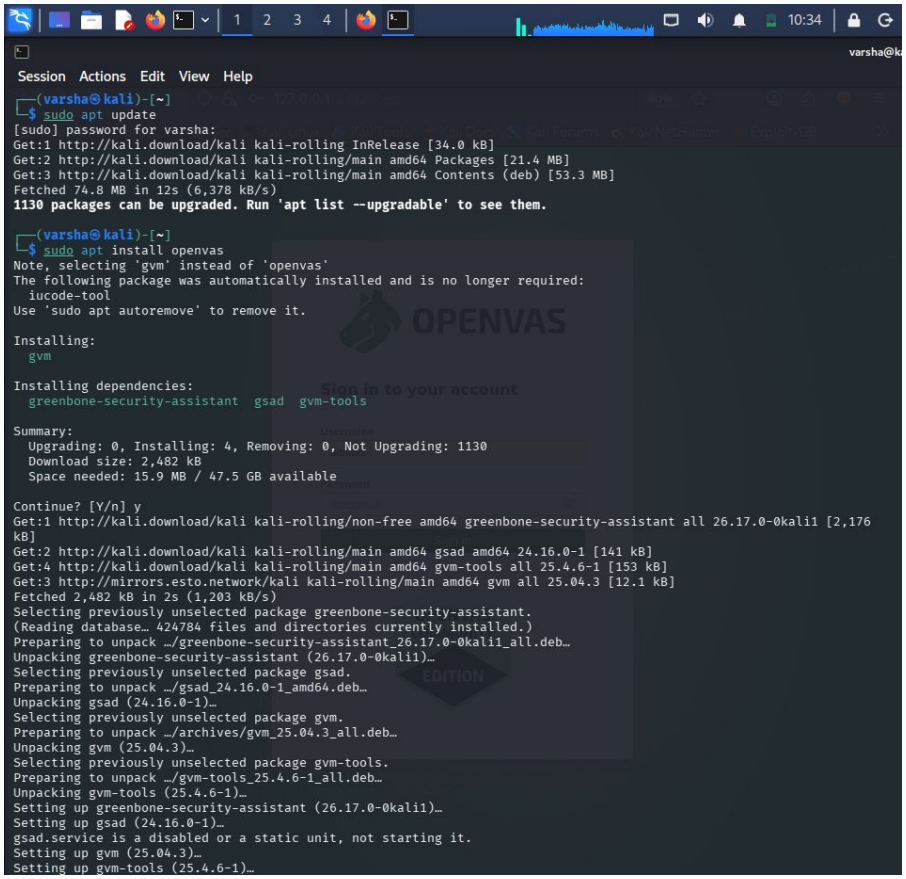


Figure 66. Installing OpenVAS

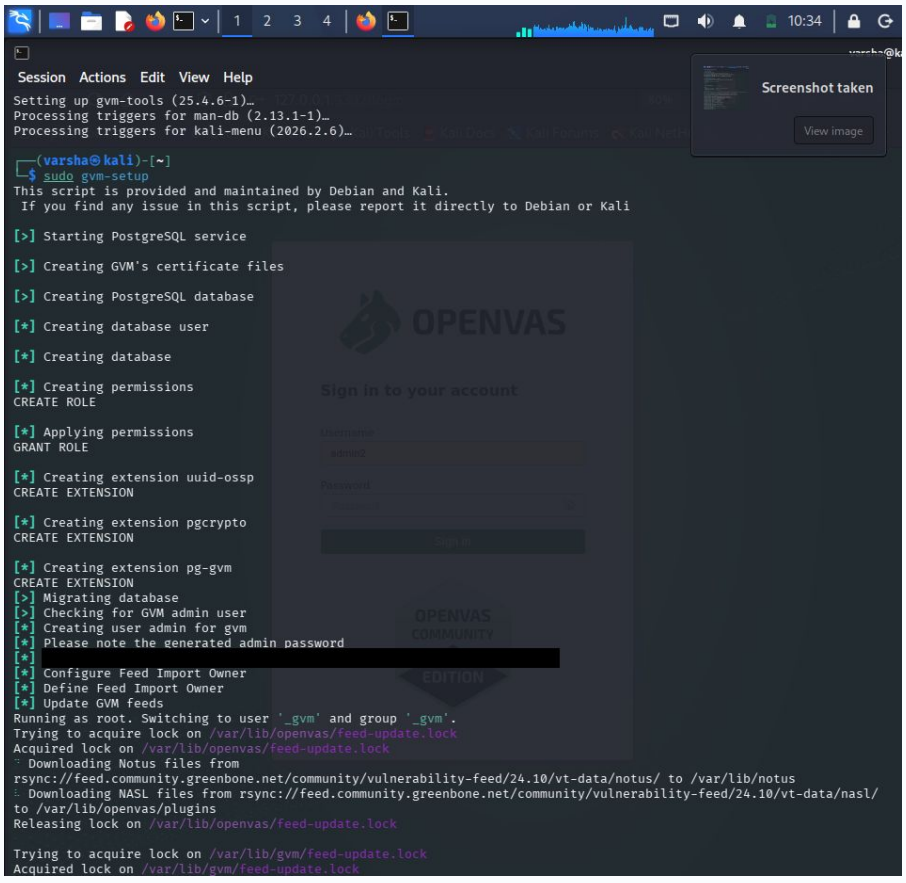


Figure 67. gvm-setup output (generated password redacted)

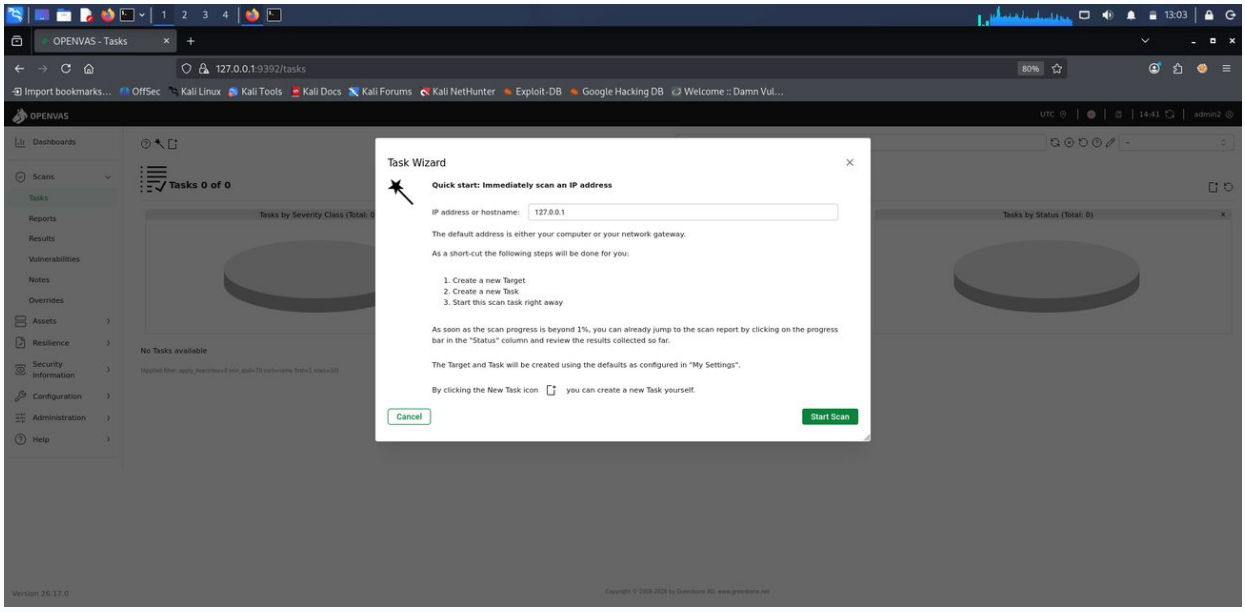


Figure 68. Task Wizard: scan of 127.0.0.1

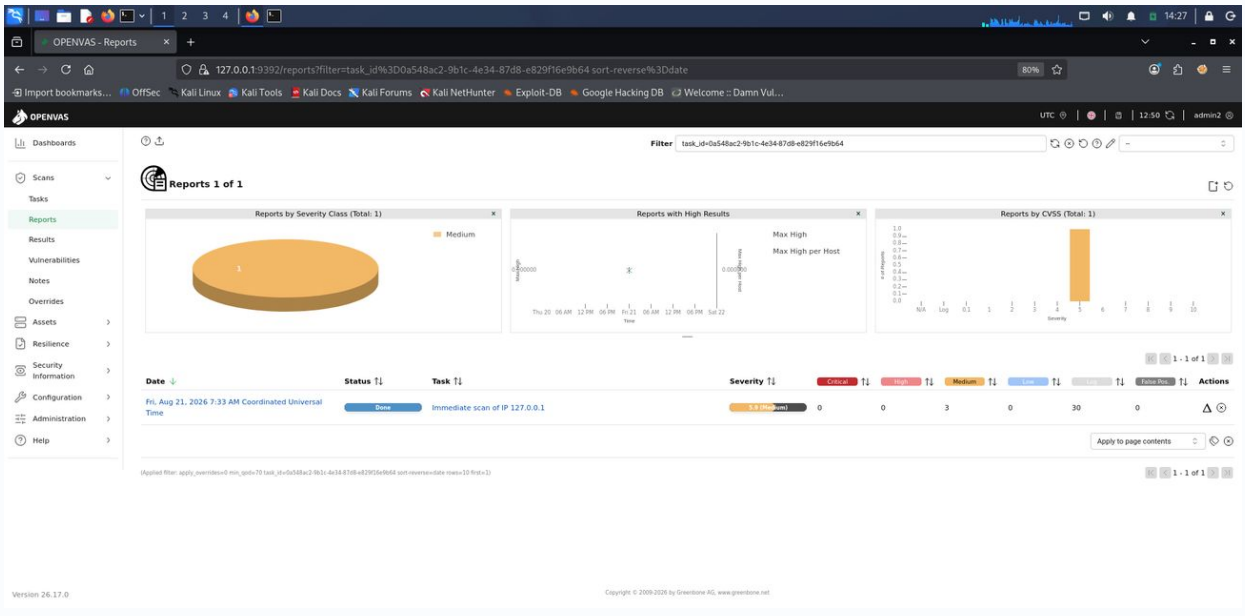


Figure 69. Scan report: severity summary (1 Medium, 3 Low)

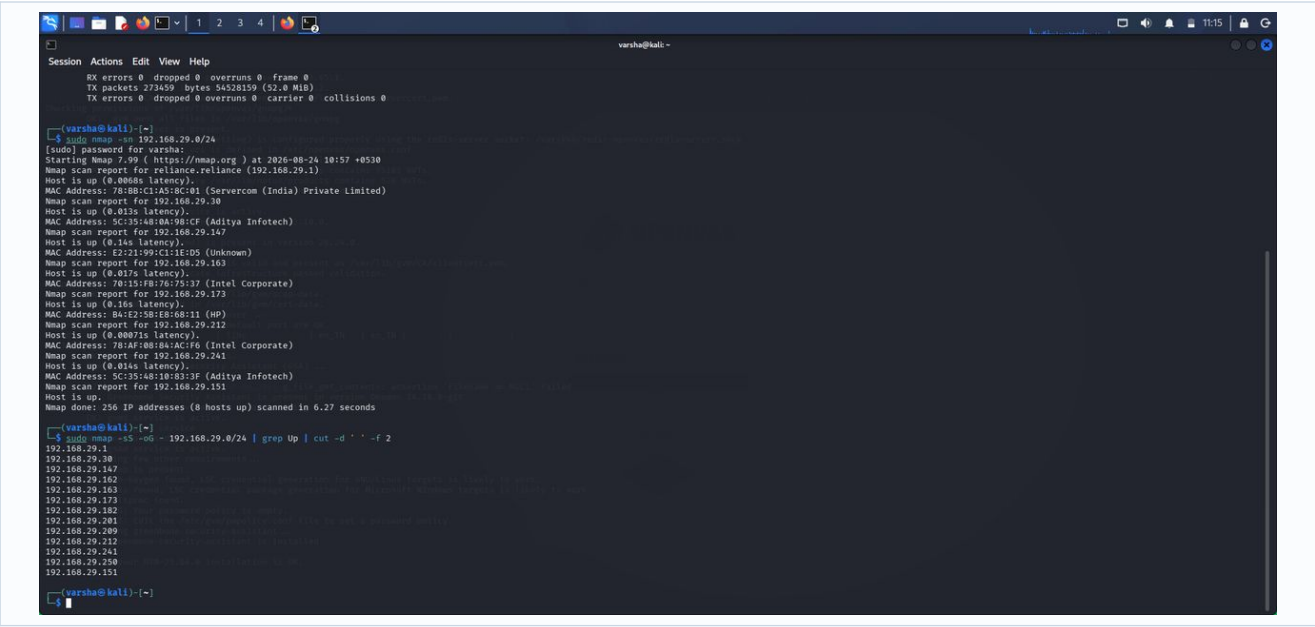


Figure 70. Nmap host discovery of the local network (8 hosts up)

16. OWASP Juice Shop challenges

Worked through the 16 challenges below and recorded the vulnerability class and fix for each. The score-board screenshot was taken partway through and shows 14 of 186 challenges solved (12% of hacking challenges) at that moment:

Challenge	Vulnerability class	Key mitigation
Score Board	Security through obscurity	Real authorization, not hidden URLs
Confidential Document	Sensitive data exposure	Server-side access controls
Missing Encoding	Improper input validation	Validate input, encode output
Zero Stars	Improper input validation	Enforce valid ranges server-side
Admin Section	Broken access control	Server-side role checks
Login MC SafeSearch	Credential exposure	Protect credentials, strong auth
DOM XSS	Cross-site scripting	Encode/sanitize, avoid unsafe DOM sinks
Error Handling	Information disclosure	Generic errors, detailed server logs
Outdated Allowlist	Improper redirect validation	Maintain and validate allowlists
Repetitive Registration	Improper input validation	Complete server-side validation
Meta Geo Stalking	Sensitive data exposure	Avoid security questions, use reset tokens
Exposed Credentials	Sensitive data exposure	Secrets management, no hardcoded credentials
Security Policy	Security policy issue	Publish a disclosure process
Privacy Policy	Privacy / information disclosure	Clear, accessible privacy policy
Login Admin	Injection / auth bypass	Parameterized queries, strong auth
Five-Star Feedback	Broken access control	Backend authorization checks

Evidence from the lab

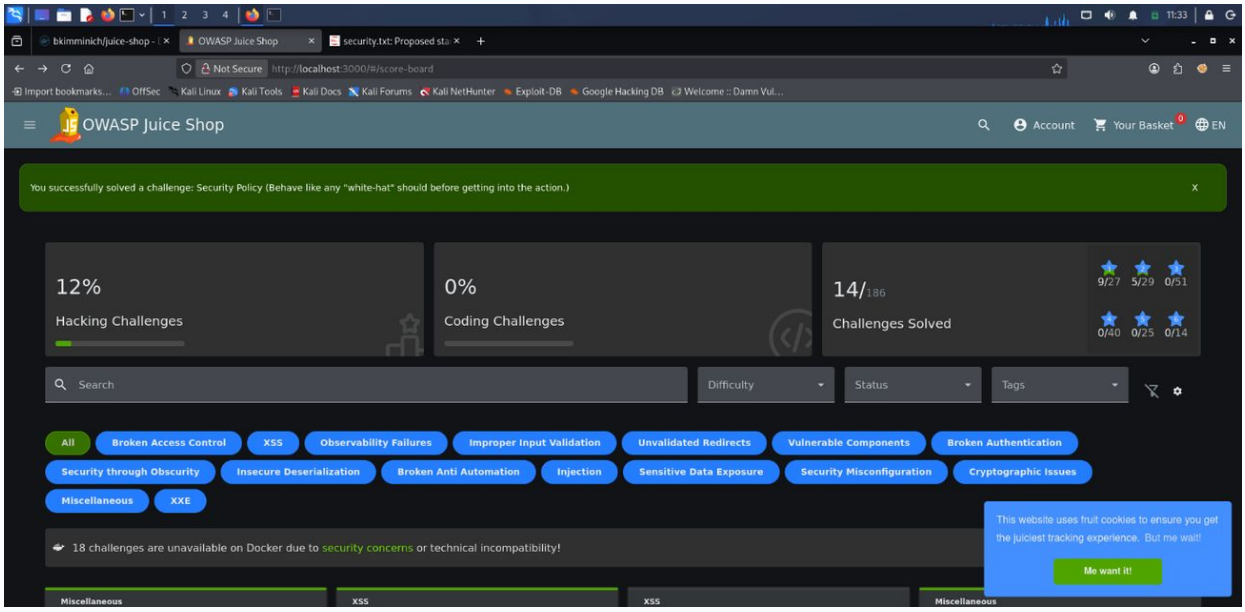


Figure 71. OWASP Juice Shop score board: 14 of 186 challenges solved at this point (Security Policy just completed)

17. Incident report: PHP2Shell

Critical remote code execution vulnerabilities in PHP

Field	Detail
Report date	25 October 2026
Incident type	Software vulnerability / Remote Code Execution (RCE)
Severity	Critical (CVSS v4.0 score 9.2 for both)
Affected technology	PHP runtime
CVE IDs	CVE-2026-109618, CVE-2026-109619
Public disclosure	21 October 2026

Executive summary

PHP2Shell is the name used for two critical vulnerabilities in the PHP runtime that can allow an unauthenticated attacker to execute code on a vulnerable web server. Both stem from memory-handling flaws: one involves memory corruption when PHP runs out of memory during a request, the other involves stale references to memory that has already been released.

Successful exploitation could let attackers compromise web servers, access sensitive information, modify website content, or disrupt services. The PHP team has released fixed versions. Organizations should identify affected systems, patch, verify the fix, and investigate for signs of compromise.

Background

PHP is a server-side language that powers a very large share of websites and web applications. A flaw in the PHP runtime is more serious than a bug in a single site, because it affects the software that executes the code of every application running on it.

Vulnerability details

3.1 CVE-2026-109618: out-of-memory handling

When PHP cannot allocate memory during request processing, it may interrupt the current operation and leave internal request data partially initialized. During cleanup, PHP can then mishandle that data, causing **memory corruption**. An attacker who triggers this condition may be able to run arbitrary code on the server.

3.2 CVE-2026-109619: use-after-free / stale state

A **use-after-free** occurs when a program keeps using memory after releasing it. Here, stale internal request state can point to memory that is no longer valid, which under certain conditions allows unsafe memory access and potentially code execution.

Both are rated **Critical** with a CVSS v4.0 score of **9.2**.

How an attack could work

1. **Identify a target** running an affected PHP version, typically an internet-facing web server.
2. **Send a crafted HTTP request** designed to trigger the faulty memory handling.
3. **Trigger the flaw**, causing memory corruption or use of invalid internal state.
4. **Gain code execution** with the privileges of the PHP or web-server process.
5. **Potential compromise:** read sensitive files, steal application credentials, alter site content, install malware, or disrupt services.

This is a potential attack sequence. A vulnerable server is not necessarily a compromised one.

Potential impact

- **Unauthorized access** to server resources
- **Data exposure:** credentials, application secrets, customer data
- **Website modification**

- **Malware installation** or persistence
- **Service disruption**
- **Lateral movement:** a compromised server can be used to attack other systems

Actual impact depends on configuration, privileges, architecture, and other security controls.

Affected systems

Potentially affected environments include:

- PHP-FPM deployments
- Apache with `mod_php` (flagged by the PHP project as particularly concerning)
- PHP-CGI deployments
- FrankenPHP and LiteSpeed environments
- Web applications and third-party products that bundle a vulnerable PHP runtime

Teams should check the PHP runtime each application **actually uses**, not rely only on asset inventories.

Evidence of exploitation attempts

Amazon Threat Intelligence reported that honeypot telemetry recorded a PHP2Shell exploitation attempt on 17 October 2026, before public disclosure, with more attempts and proof-of-concept activity around the disclosure date. An exploitation *attempt* does not by itself prove that a real production server was compromised, so organizations should review their own logs.

Detection and investigation

Look for:

- Unusual entries in web-server access and error logs
- Unexpected PHP process crashes or abnormal resource use
- Unusual child processes spawned by PHP or the web server
- Unexpected file creation or modification in web directories
- Suspicious outbound network connections
- Unexpected access to application secrets or sensitive files

EDR and SIEM tools can help, but a single odd request or process is not enough to confirm exploitation. Assess findings in context.

Mitigation and remediation

The primary fix is to **upgrade PHP** to a fixed release, or install the equivalent vendor security update. Fixed versions released on 21 October 2026:

PHP branch	Fixed version
8.5	8.5.12
8.4	8.4.27
8.3	8.3.36
8.2	8.2.35

Response checklist

1. Inventory all servers, applications, and appliances running PHP.
2. Determine the real installed runtime version and check the relevant vendor advisory.
3. Prioritize exposed, internet-facing systems.
4. Apply the security updates.
5. Restart or redeploy services where required.
6. Verify the live application uses the corrected runtime.
7. Review logs for suspicious activity before and after patching.
8. If evidence of compromise is found, follow incident response procedures.

Unsupported PHP branches need particular attention because they may not receive upstream fixes. Vendor-packaged and backported fixes should be confirmed with the vendor.

Preventive measures

- Maintain an accurate inventory of software and runtime versions
- Patch promptly and avoid unsupported versions
- Run web-server processes with least privilege
- Monitor activity and retain logs
- Restrict unnecessary network access
- Keep tested backups and a rehearsed incident response plan
- Run regular vulnerability assessments and configuration reviews

A web application firewall can add a layer of protection but is not a substitute for installing the official patches.

Conclusion

PHP2Shell shows how flaws in widely deployed software can put many sites at risk at once. The key lessons are accurate asset inventory, fast patching, continuous monitoring, and a ready incident response process.

References

1. PHP Foundation: PHP security releases, 21 October 2026. <https://www.php.net/>
2. Canadian Centre for Cyber Security: Critical vulnerabilities impacting PHP. <https://www.cyber.gc.ca/en/alerts-advisories/al26-026-critical-vulnerabilities-impacting-php-cve-2026-109618-cve-2026-109619>
3. Amazon Web Services: PHP2Shell exploitation observed before public disclosure. <https://aws.amazon.com/blogs/security/amazon-threat-intelligence-identifies-php2shell-exploitation-prior-to-public-disclosure-in-madpot/>
4. NHS England Digital: Critical vulnerabilities in PHP. <https://digital.nhs.uk/cyber-alerts/2026/cc-4875/cc-4875>
5. Plesk: PHP2Shell vulnerability and remediation guidance. <https://support.plesk.com/hc/en-us/articles/44397729587351-Vulnerability-CVE-2026-109618-and-CVE-2026-109619-PHP2Shell-Remote-code-execution-in-PHP-shipped-with-Plesk>

Prepared for cybersecurity awareness and technical research. Details reflect information available at the report date; check vendor advisories for updates.

Skills demonstrated

Network reconnaissance · Service and version enumeration · Vulnerability validation (Metasploit) · Web application testing against the OWASP Top 10 · Intercepting and automating requests with Burp Suite · Vulnerability scanning and reporting (OpenVAS/GVM) · Packet analysis (Wireshark) · Writing remediation-focused findings · Security incident research and reporting